
Implementation of Image Processing



Achmad Fauzi Bagus Firmansyah (achmad.firmansyah@bps.go.id)

Direktorat Pengembangan Metodologi Sensus dan Survei, Badan Pusat Statistik

PUT YOUR QUESTION HERE

Join at
slido.com
#3860 036





<https://github.com/achmfirmansyah/ImageProcessing>

Visit the repository for material and code in this lecture session.



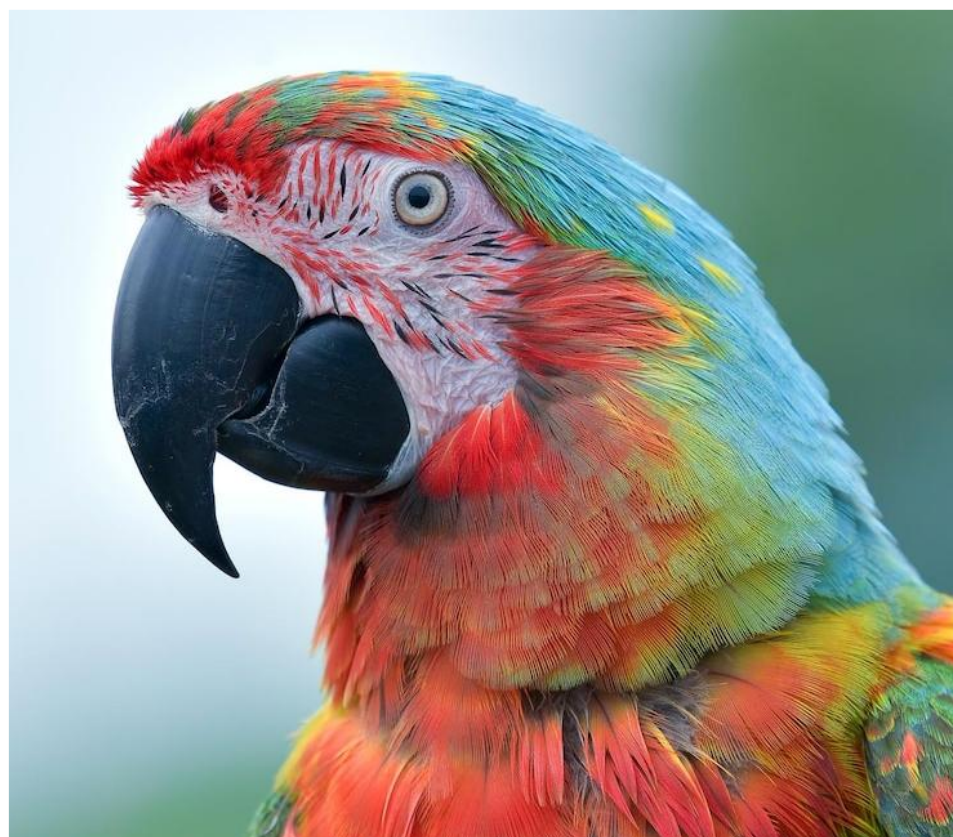
01. Recap Previous Lecture

02. Classify the Image

03. Captioning the Image

What Will we learn today ?





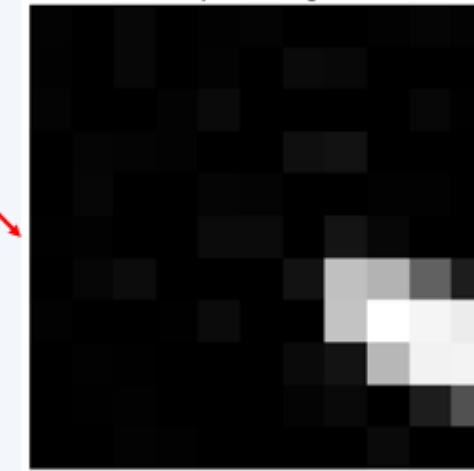
RECAP



IMAGE



Image is not a picture; **IT'S DATA**, represented by matrix of numerical value.

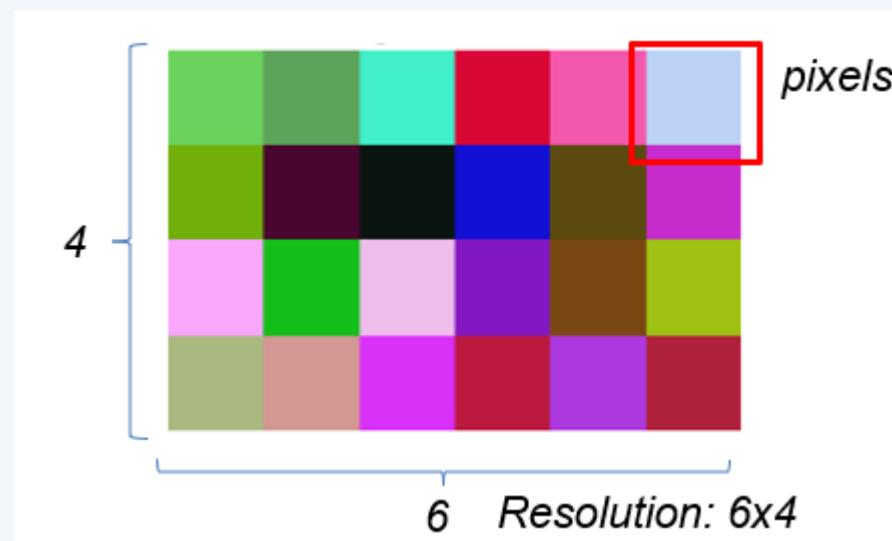


Representation of top left in Matrix Format:

[	3	0	7	0	2	4	0	0	2	5	3]
[	2	0	7	0	3	0	10	8	0	0	0]
[	4	0	0	3	10	0	0	0	0	7	1]
[	0	5	4	3	0	0	15	18	0	0	0]
[	0	6	0	0	5	4	0	0	2	2	0]
[	1	2	0	0	11	11	0	20	7	0	0]
[	0	6	12	0	0	0	18	192	180	98	29]
[	2	0	0	1	11	0	0	196	255	245	236]
[	0	2	1	0	0	0	10	19	183	242	244]
[	0	1	2	0	0	0	4	9	0	29	82]
[	0	0	3	2	0	0	0	0	10	0	0]]

Remember the concept

- **Pixels** is smallest unit of picture that compose an image.
- **Resolution** is size of the image, represented with *length x wide*.
- **Band** is layer of information of image, sometime called “channel”, eg: RGB.



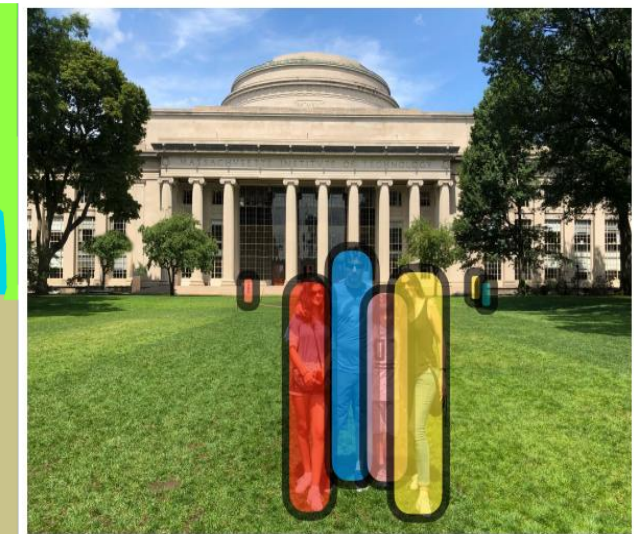
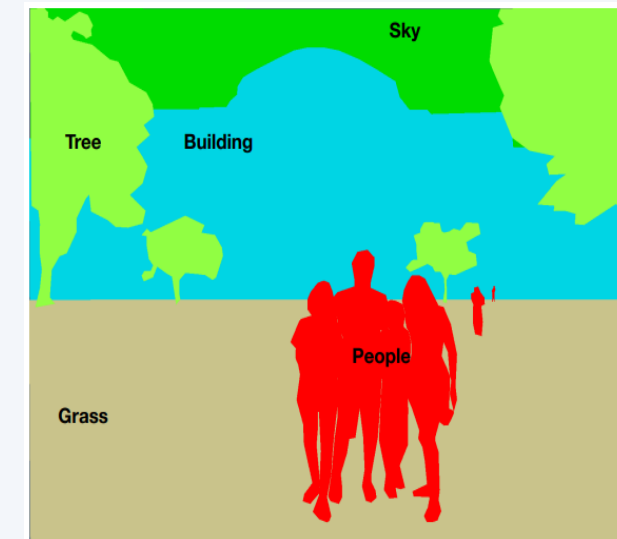
“SEEING”



In computer vision, “seeing” is the **process of converting raw image data into high-level understanding** — for example, detecting a face, identifying a car, or interpreting a handwritten digit.

From the “seeing” activities, sometimes we intend to:

1. **Classify** the image into some label
2. **Identify** all the part in image
3. **Counting** the object
4. **Recognize** the object
5. **Narrate** the object
6. **Imagine** an image giving some ideas
7. **Forecast** the next image



Spatial Filtering



1	1	1
1	1	1
1	1	1

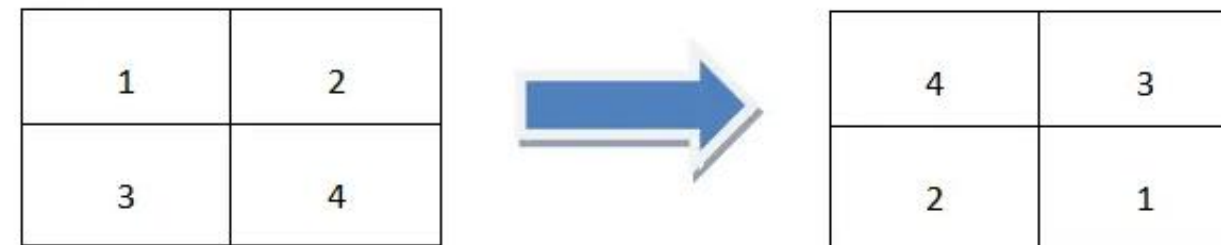
Image

1	2
3	4

Kernel

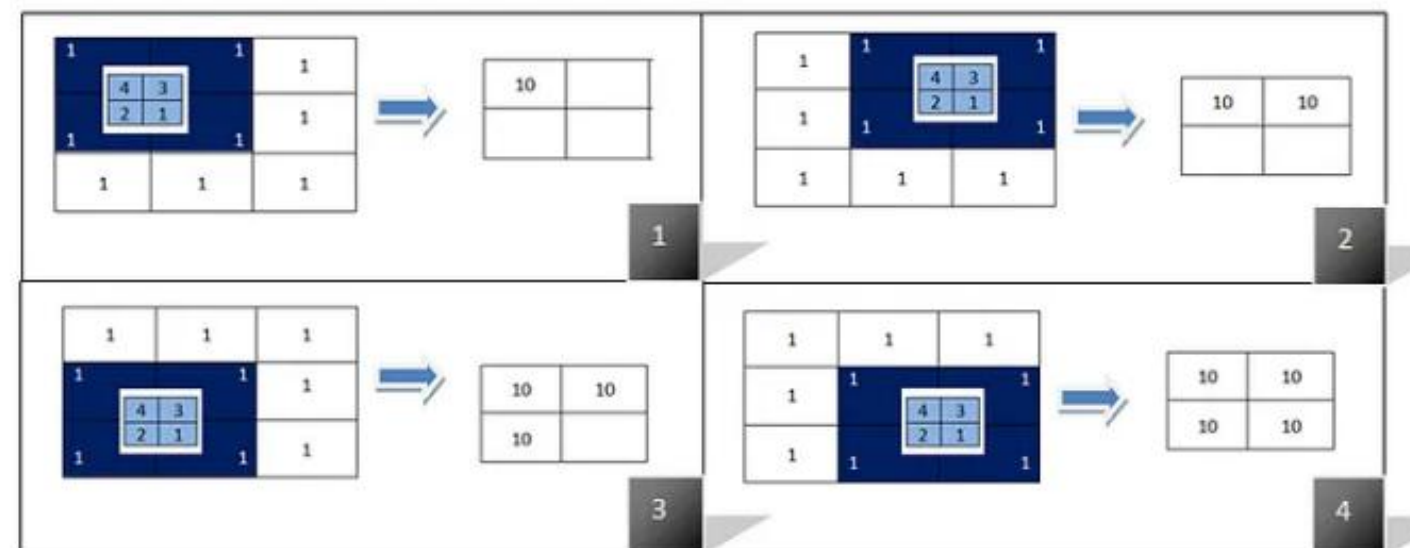
Convolution Filtering

1. Flip the filter in both dimensions (bottom to top, right to left)



Flip the filter in both dimensions

2. Apply correlation filtering



The second step which applying correlation filtering after flipping the kernel

Spatial Filtering



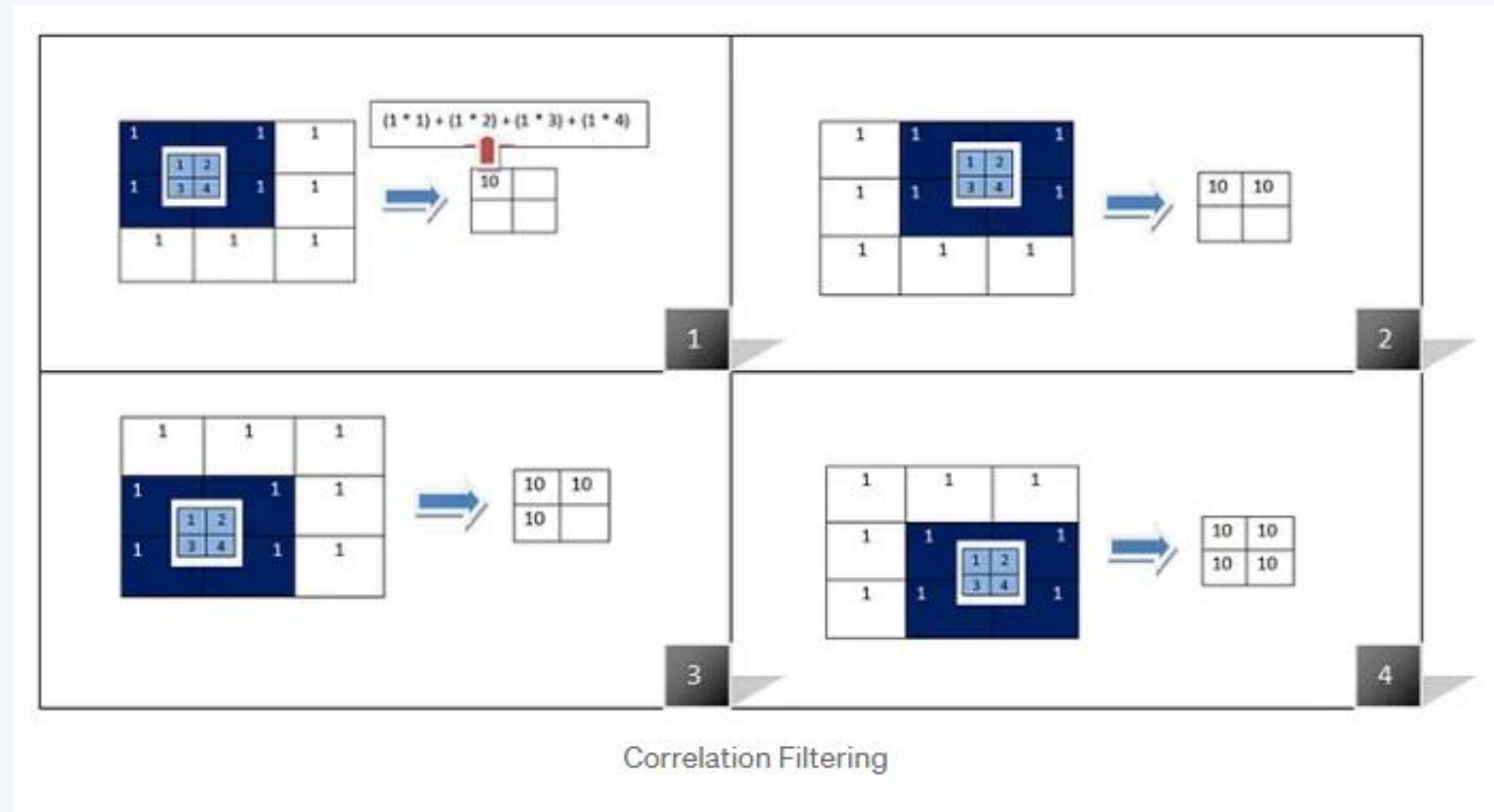
1	1	1
1	1	1
1	1	1

Image

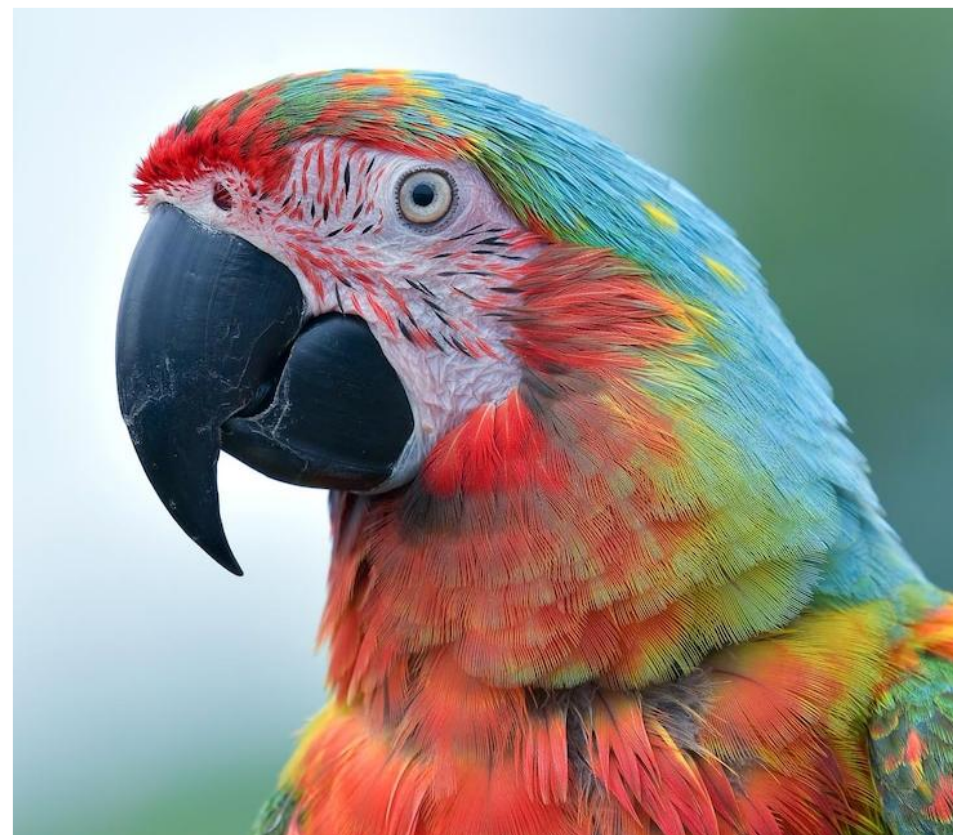
1	2
3	4

Kernel

Cross-Correlation Filtering



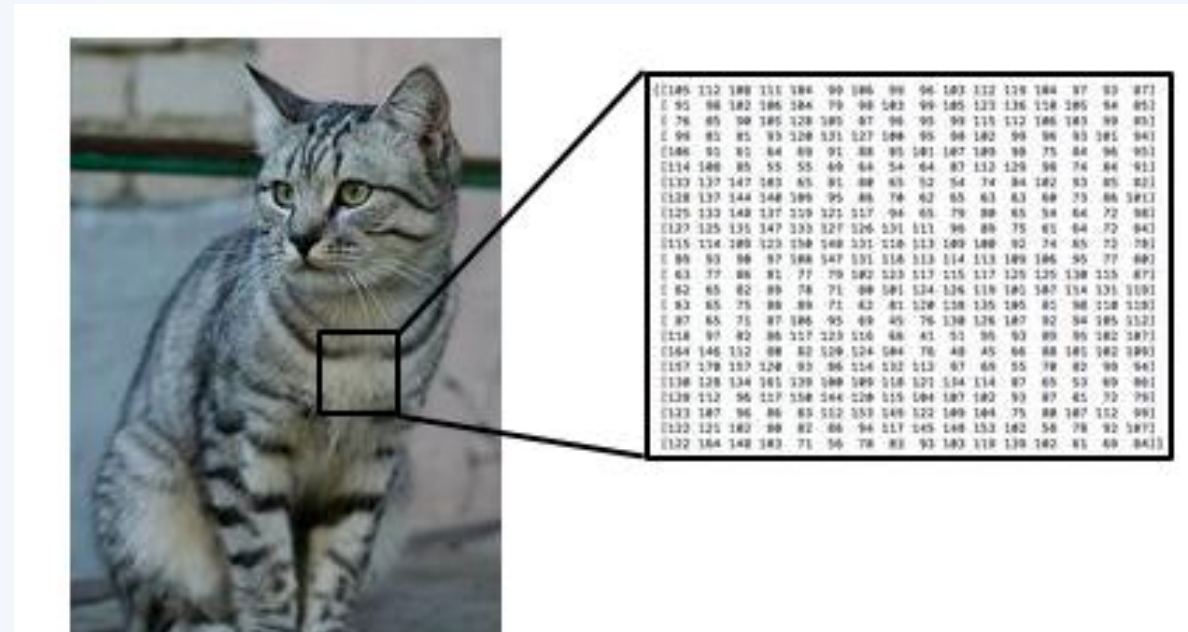
Albeit its naming, the CNN use cross correlation operation instead of convolution operation.



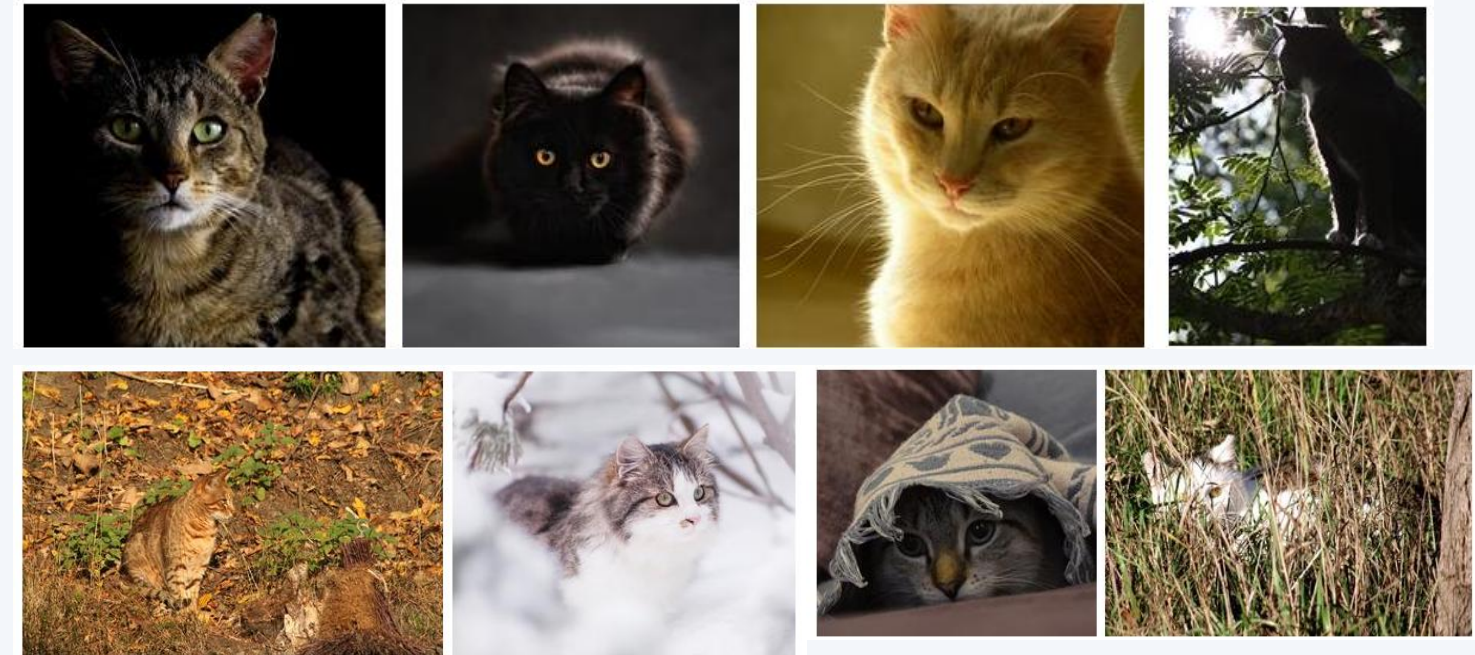
Classifying the Image

<https://cs231n.stanford.edu/index.html>

Image Classification



➔ CAT



The machine learning approach:

1. Find the training data and label
2. Get the algorithm and perform modelling
3. Evaluate and predict

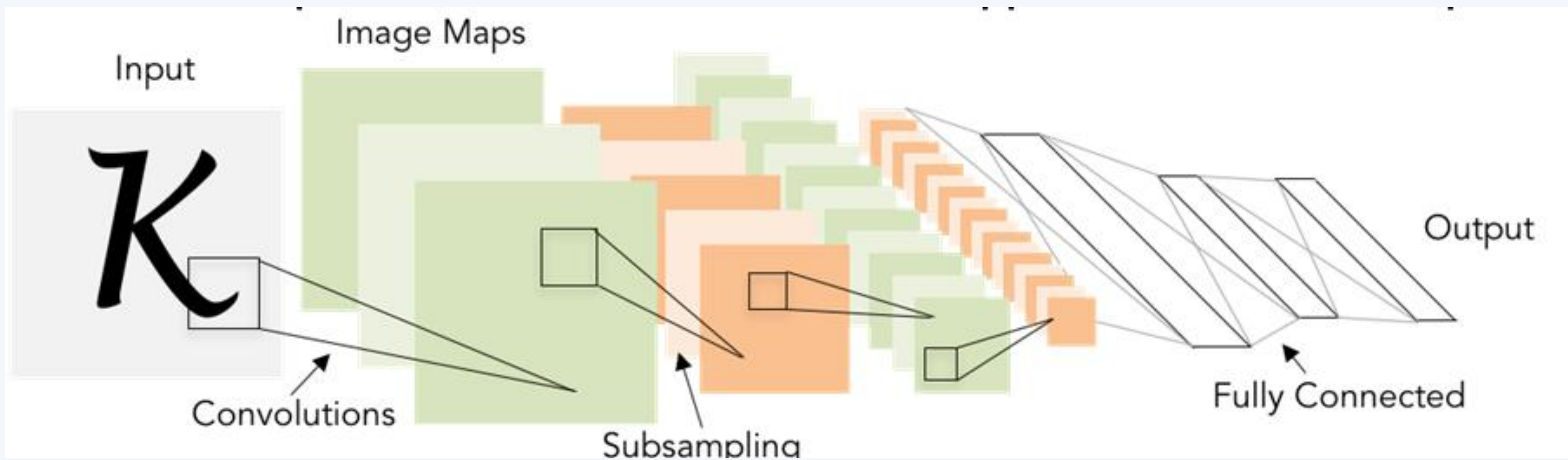
```
def train(images, labels):  
    # Machine learning!  
    return model
```

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```

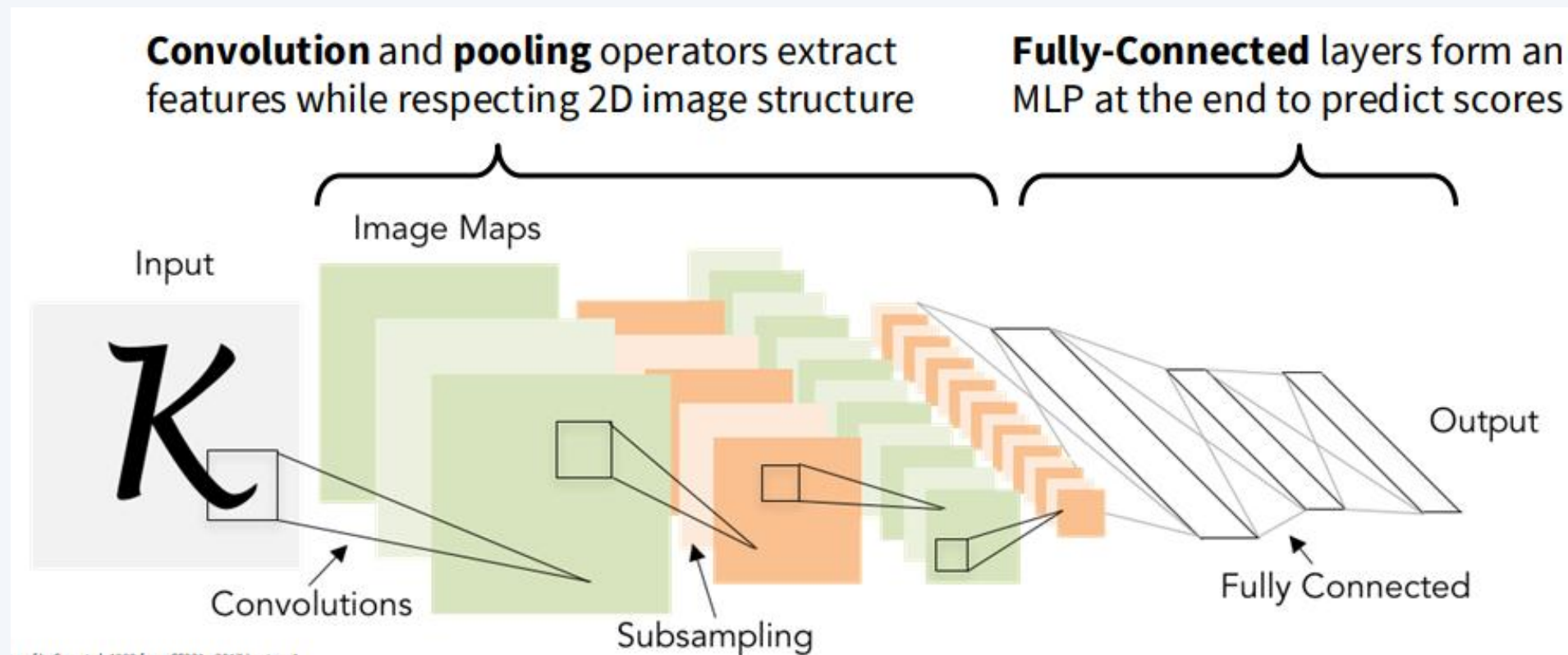
airplane
automobile
bird
cat
deer



CNN Approach



Breaking it Down!

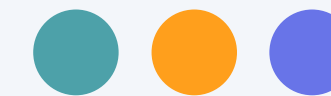


In CNN you will:

1. Ingest the images from data training with following table
2. Perform the convolution operation and activation layer
3. Down-sampling with pooling operation several times.
4. Pass the reduced/smallest part into MLP operation
5. Perform classification on MLP

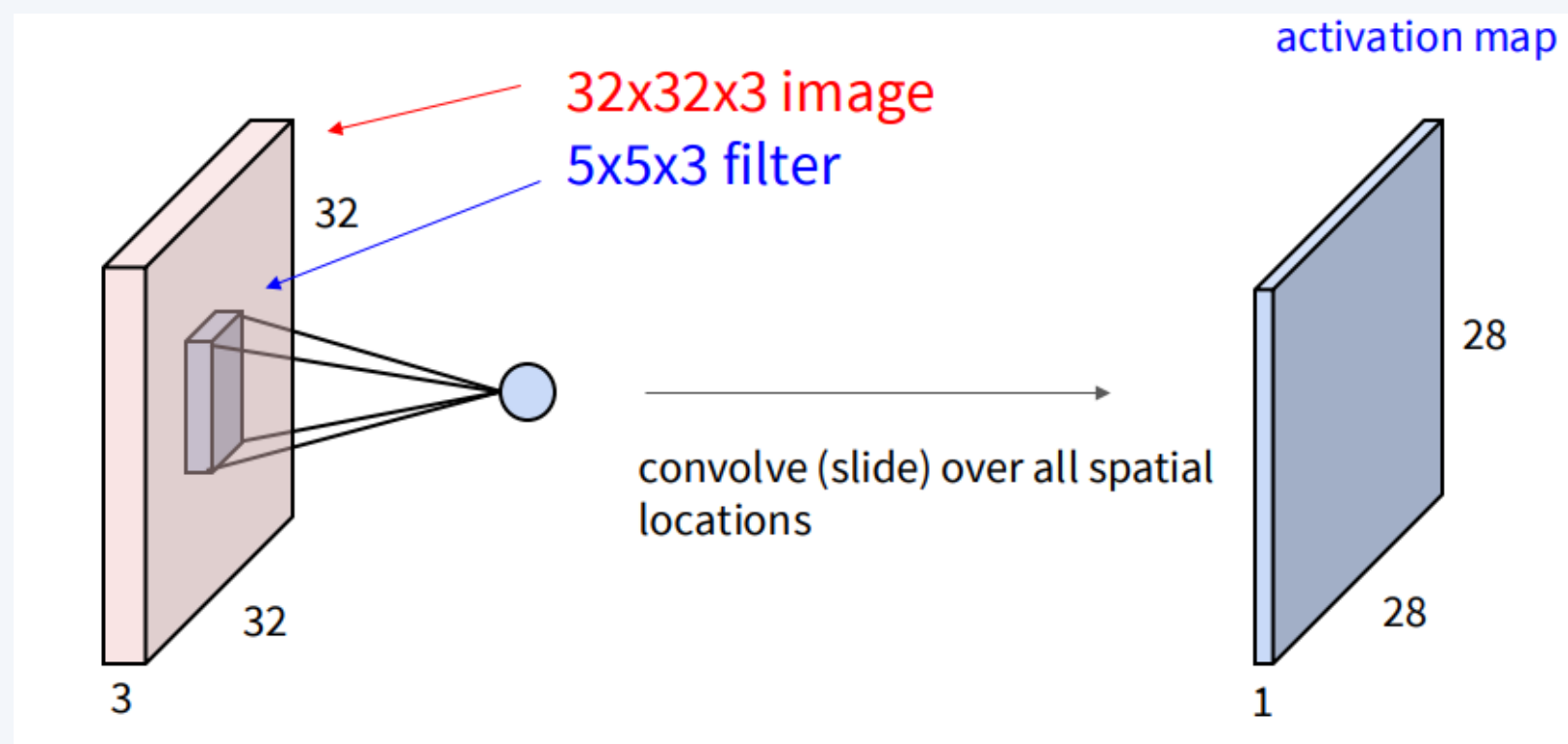
Click this link <https://poloclub.github.io/cnn-explainer/> !

ConvNet

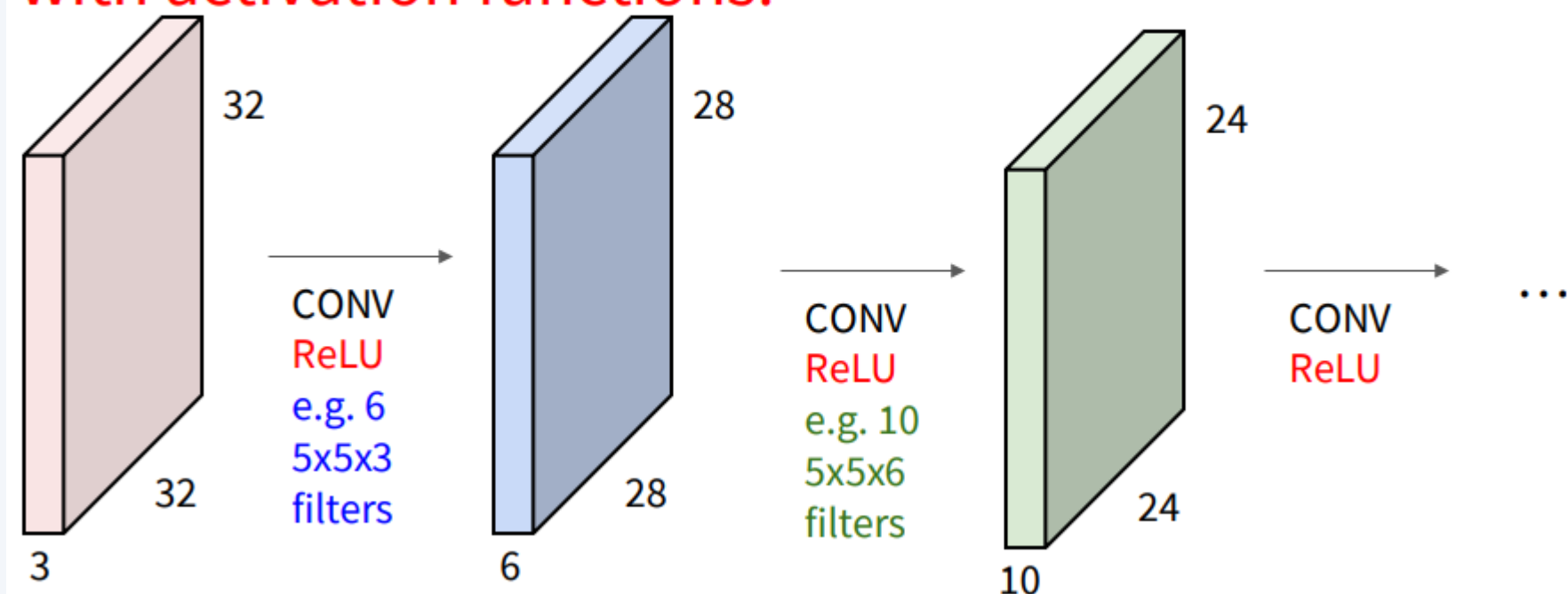


Basic Idea of ConvNet

1. Consist of Conv Layer and Activation Function (e.g. ReLU)
2. For each Conv Layer, the image will be iterated with kernels / filter using cross correlation operation



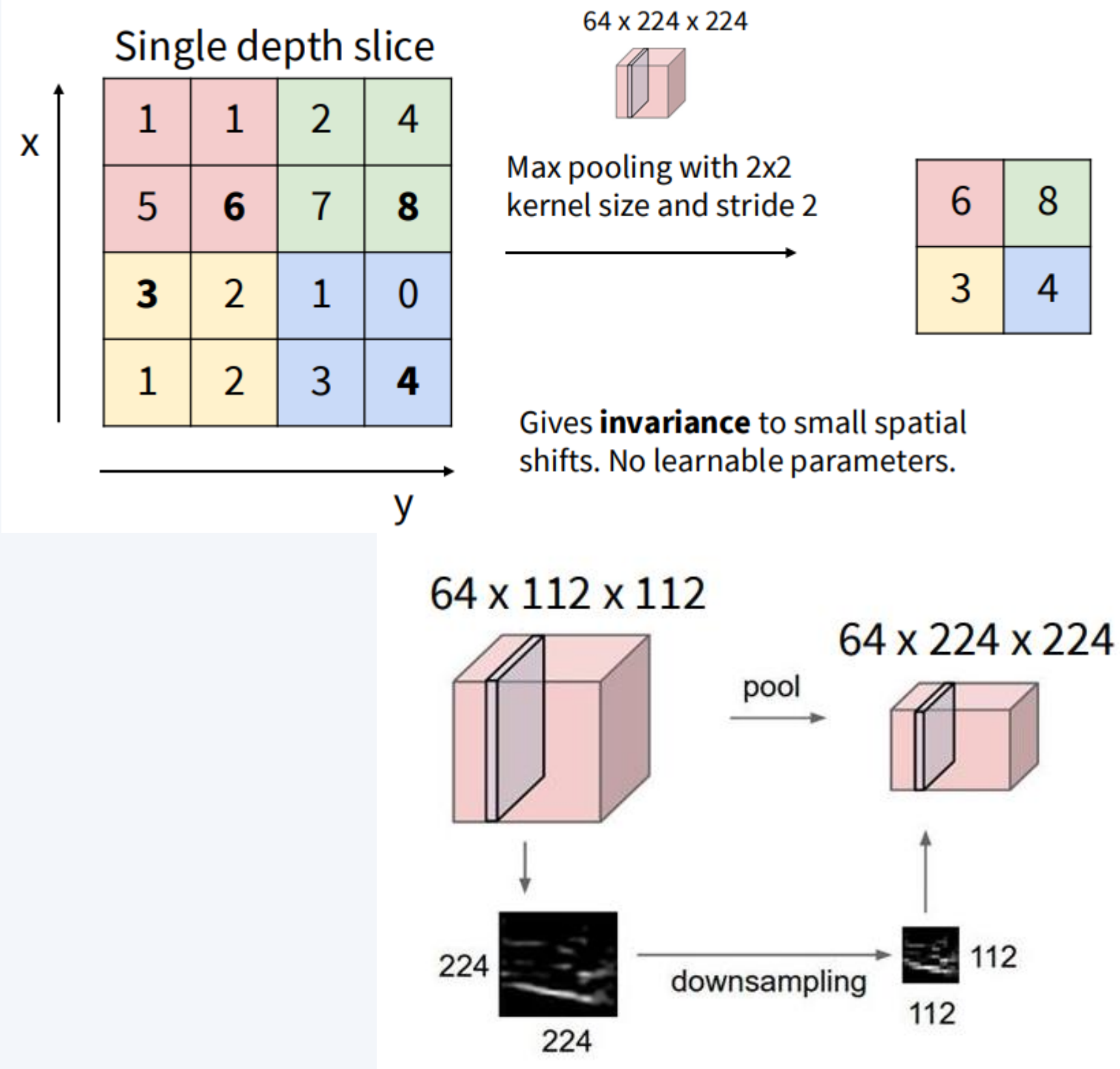
A ConvNet is a neural network with Conv layers
with activation functions!



Click this link

<https://poloclub.github.io/cnn-explainer/> !

Pooling Layer



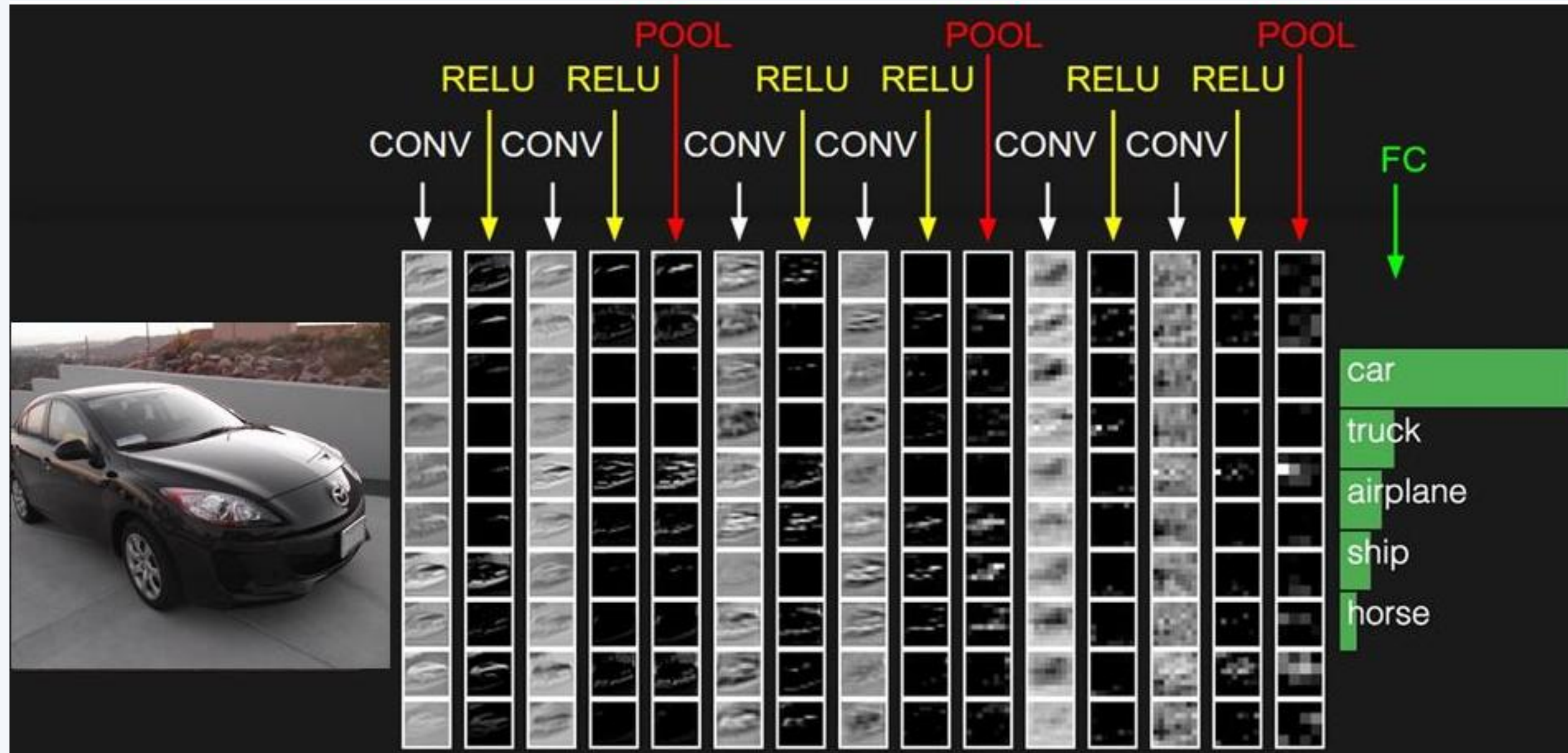
Basic Idea of ConvNet

1. Consist of Conv Layer and Activation Function (e.g. ReLU)
2. For each Conv Layer, the image will be iterated with kernels / filter using cross correlation operation
3. After finish operating the ConvNet, it would be passed to Pooling Layer to reduce the dimension/downsampling

Click this link

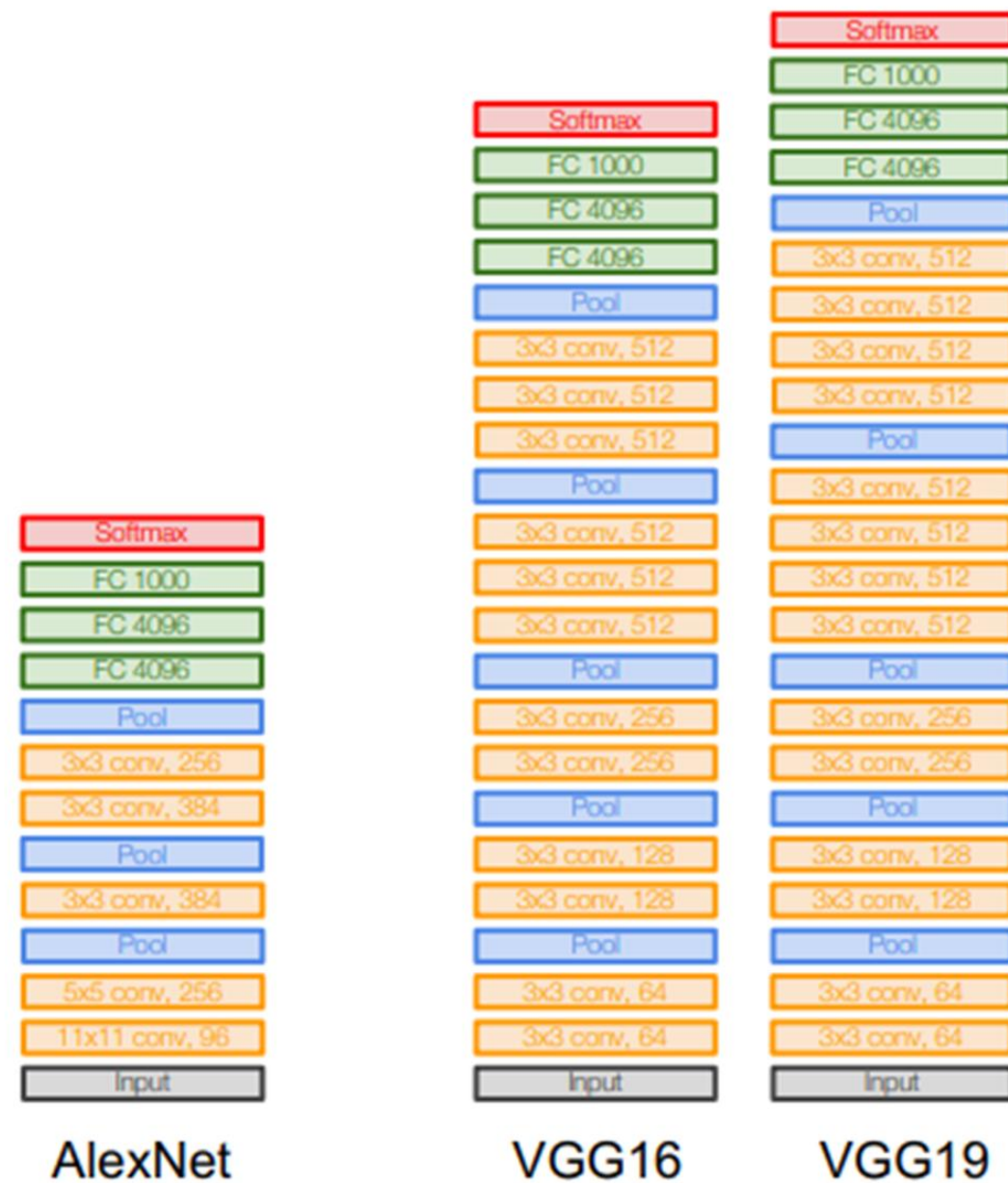
<https://poloclub.github.io/cnn-explainer/> !

More...

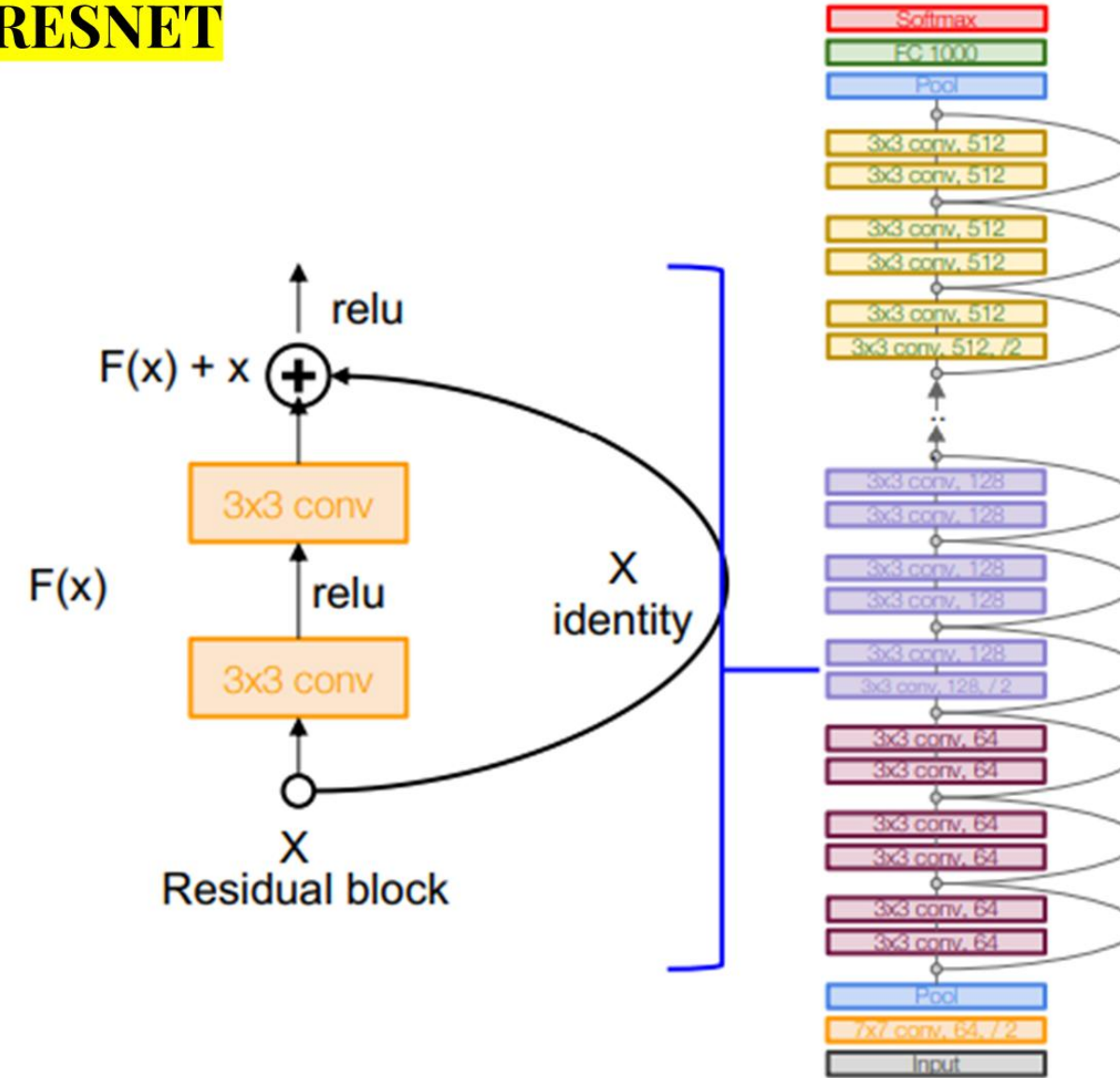


DEMO: <http://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>

Pretrained Network

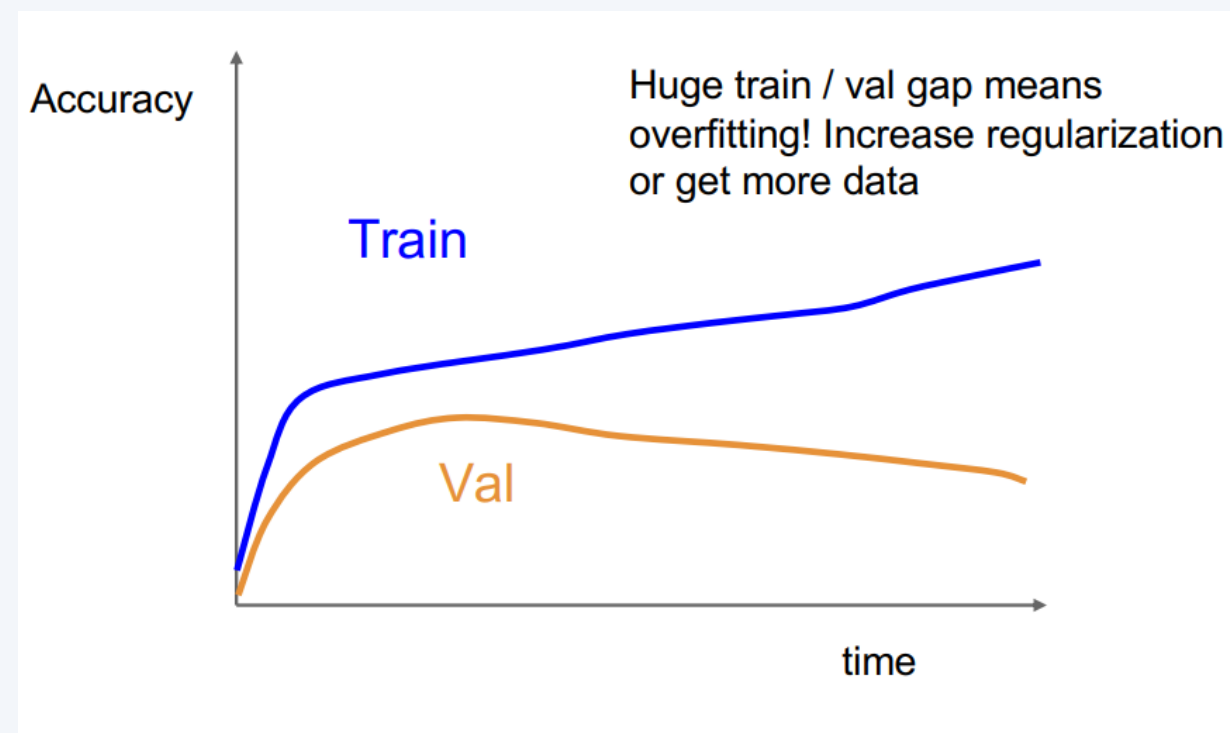
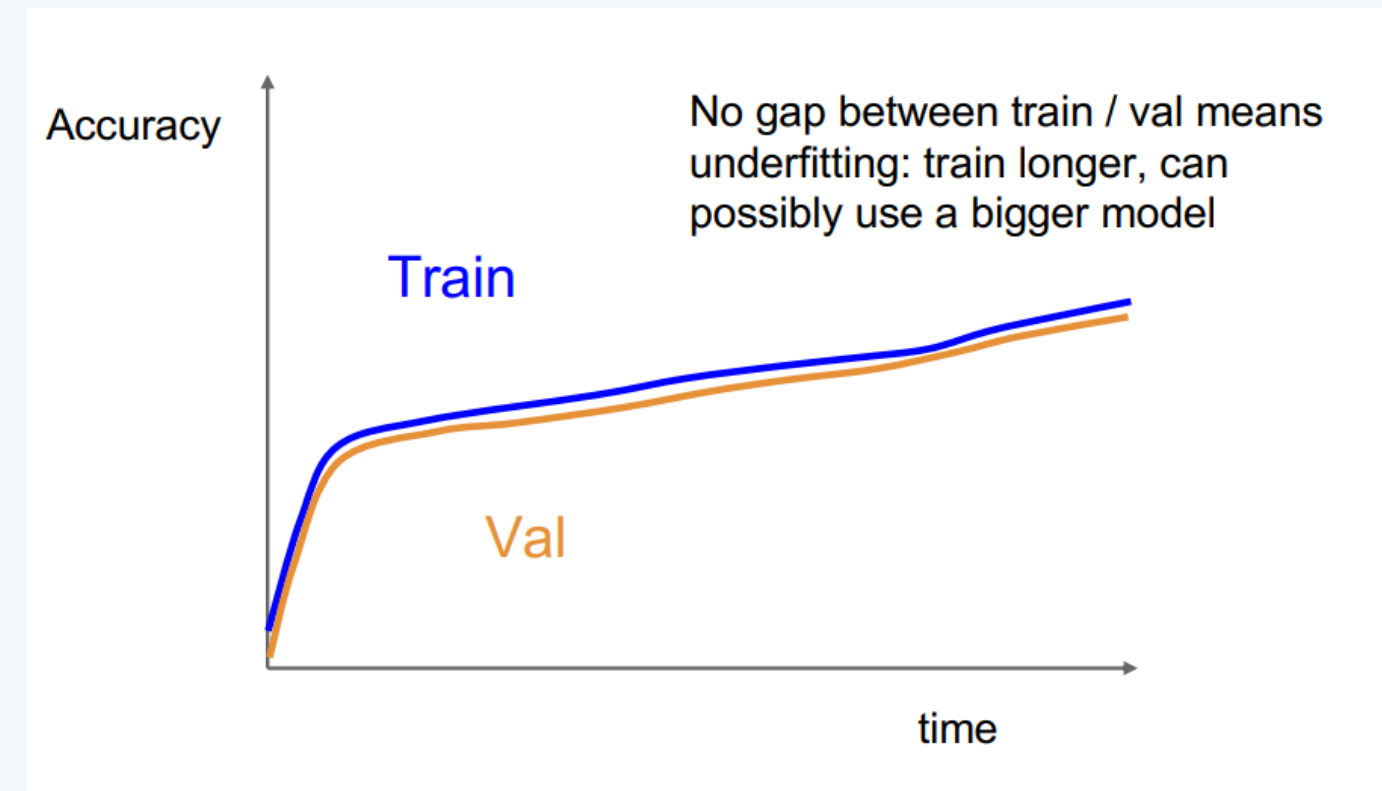
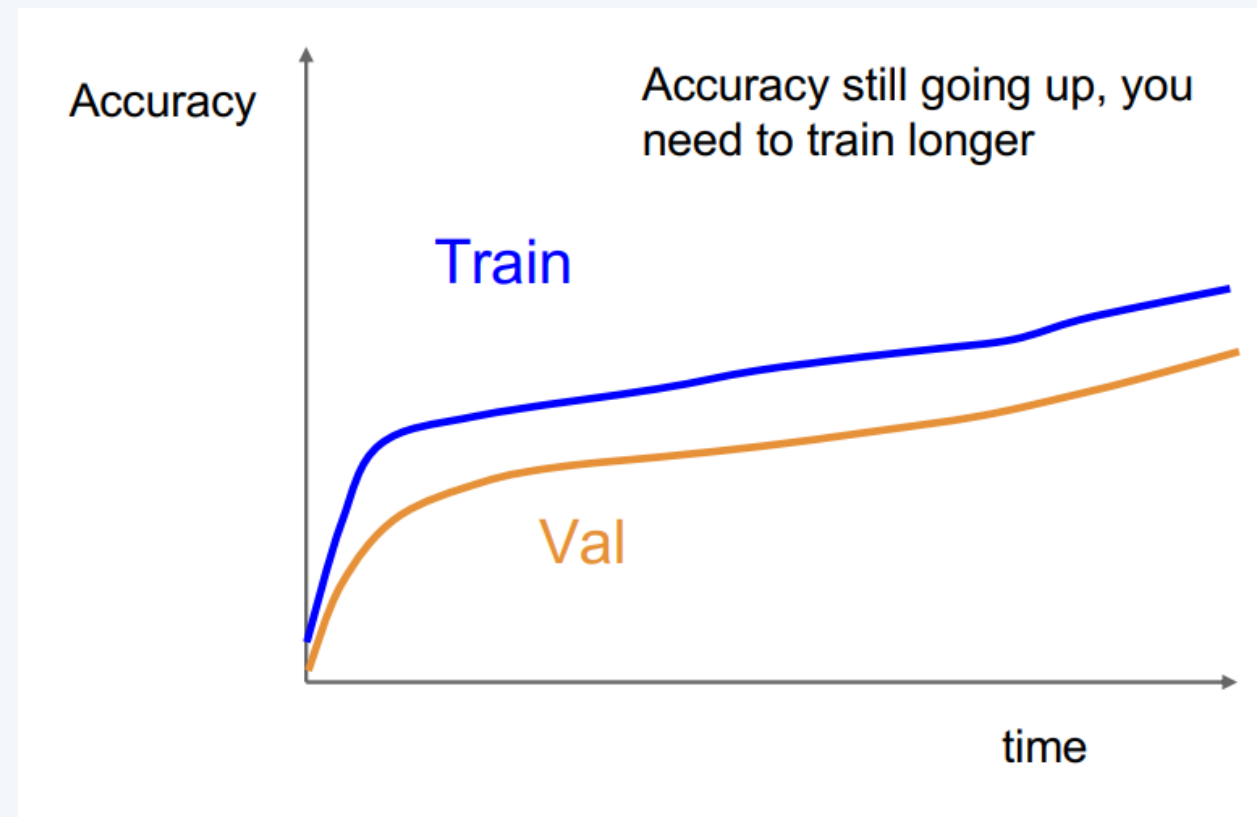


RESNET



<https://www.kaggle.com/code/deepakat002/vgg-resnet-inception-xception-comparision>

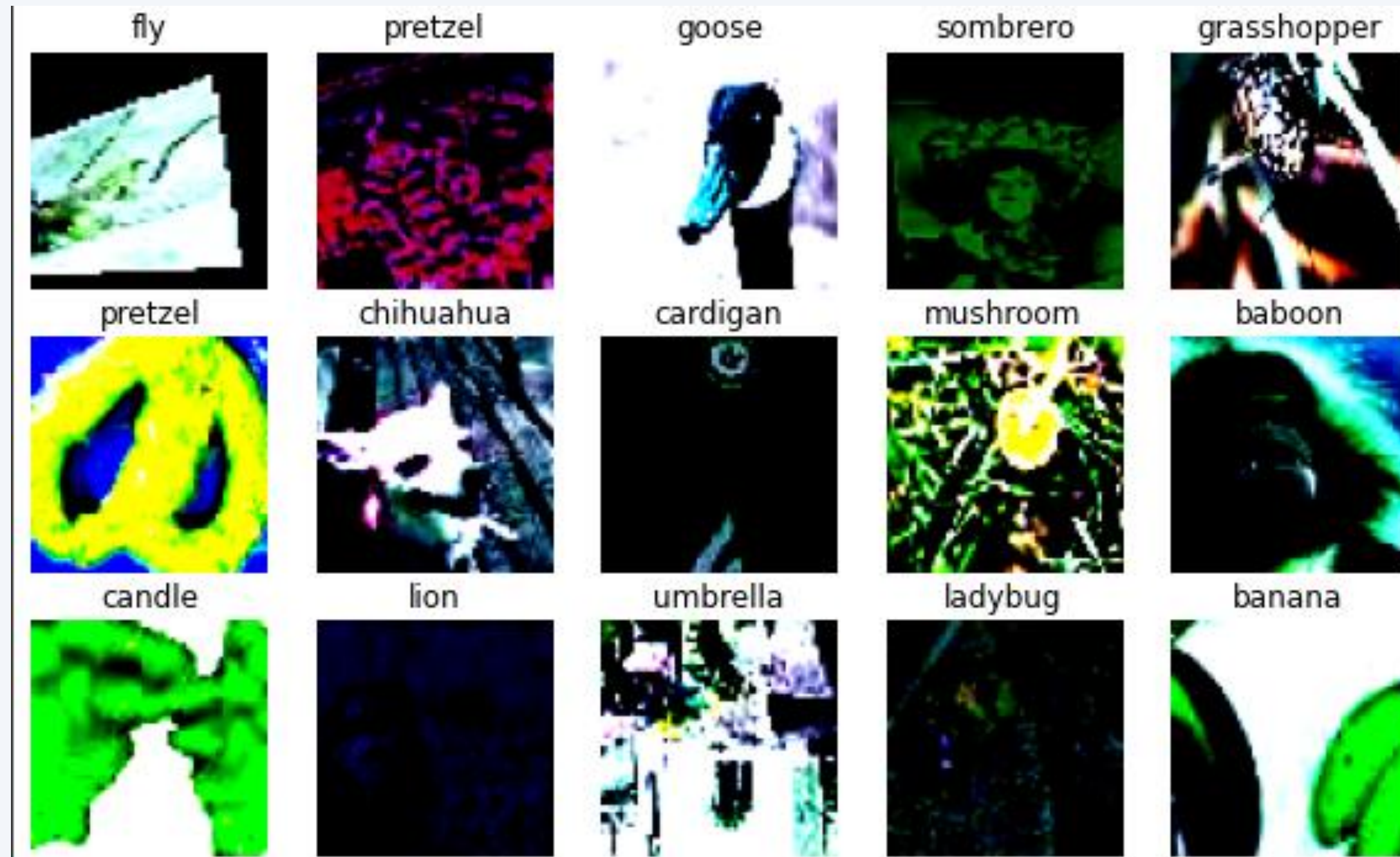
Accuracy



Stop training when...

- 1. No improved accuracy (both train and validation)**
- 2. No huge gap between them**

Data Augmentation



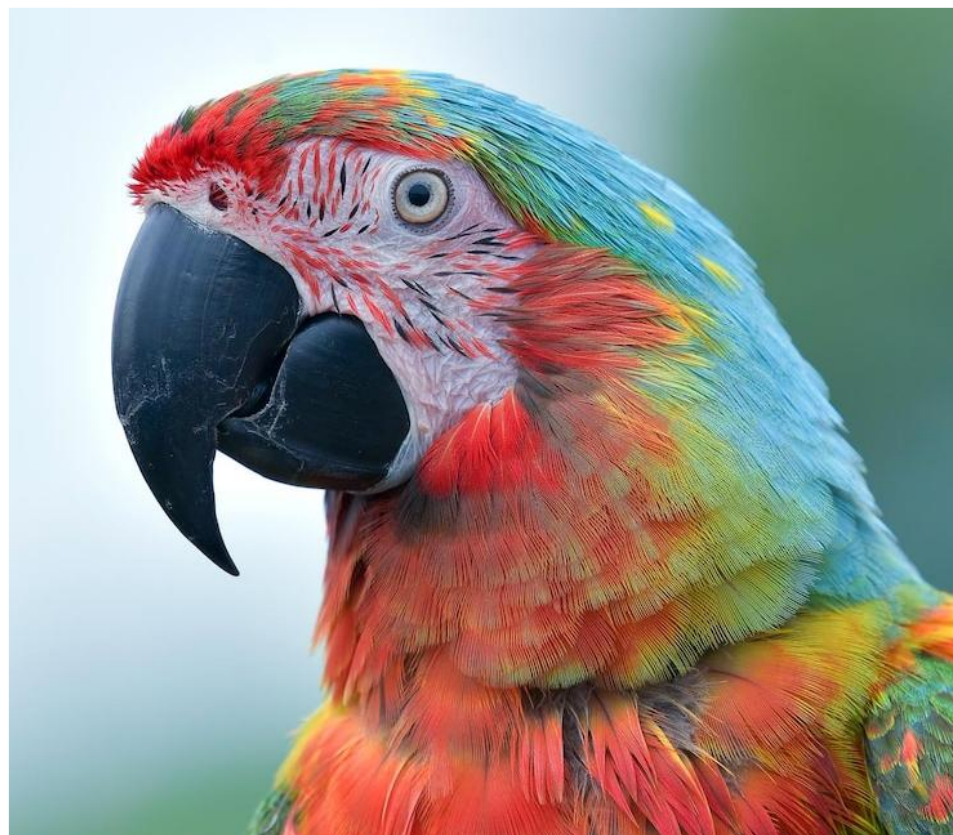
Check my work on my Master Course:

https://github.com/achmfirmanasyah/ImageProcessing/tree/main/02_Code/01_Notebook

CEK OMBAK DULU...

join my quiz.com • 896058





Captioning the Image



<https://cs231n.stanford.edu/index.html>

Image Captioning



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



Two giraffes standing in a grassy field



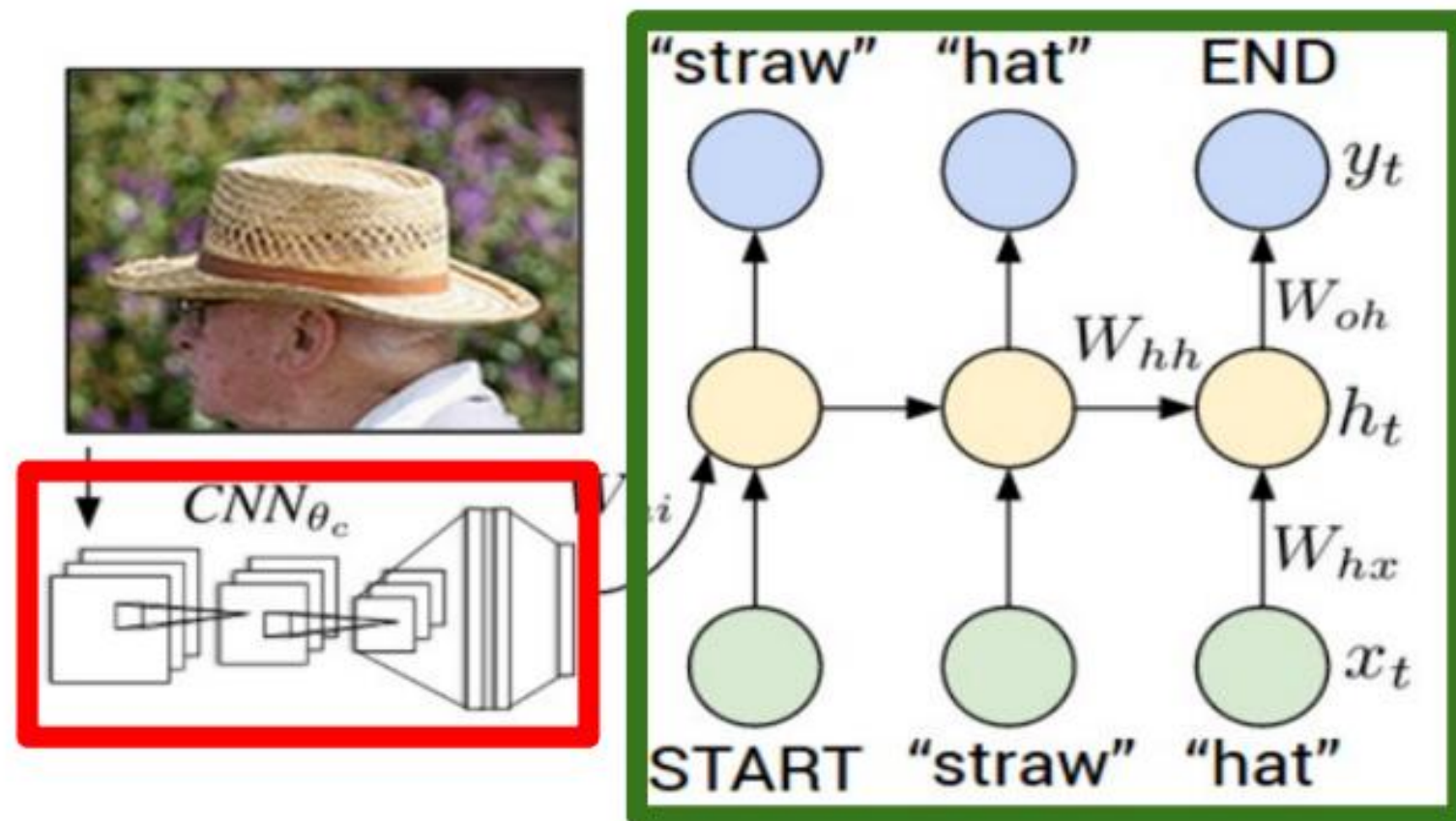
A man riding a dirt bike on a dirt track

How does it work?

Image Captioning



Recurrent Neural Network

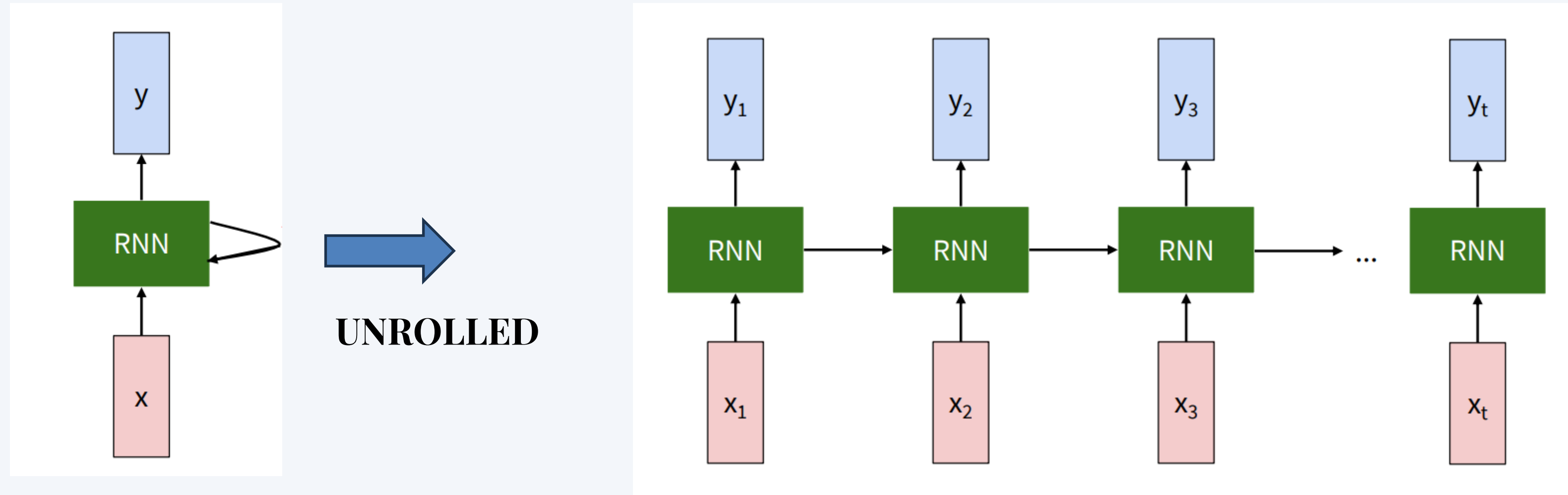


Convolutional Neural Network

In a short way:

1. Do the **CNN** part until penultimate layer
2. Transfer it to the **RNN** part

RNN? What is it?



RNN? What is it?

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

old state

input vector at some time step

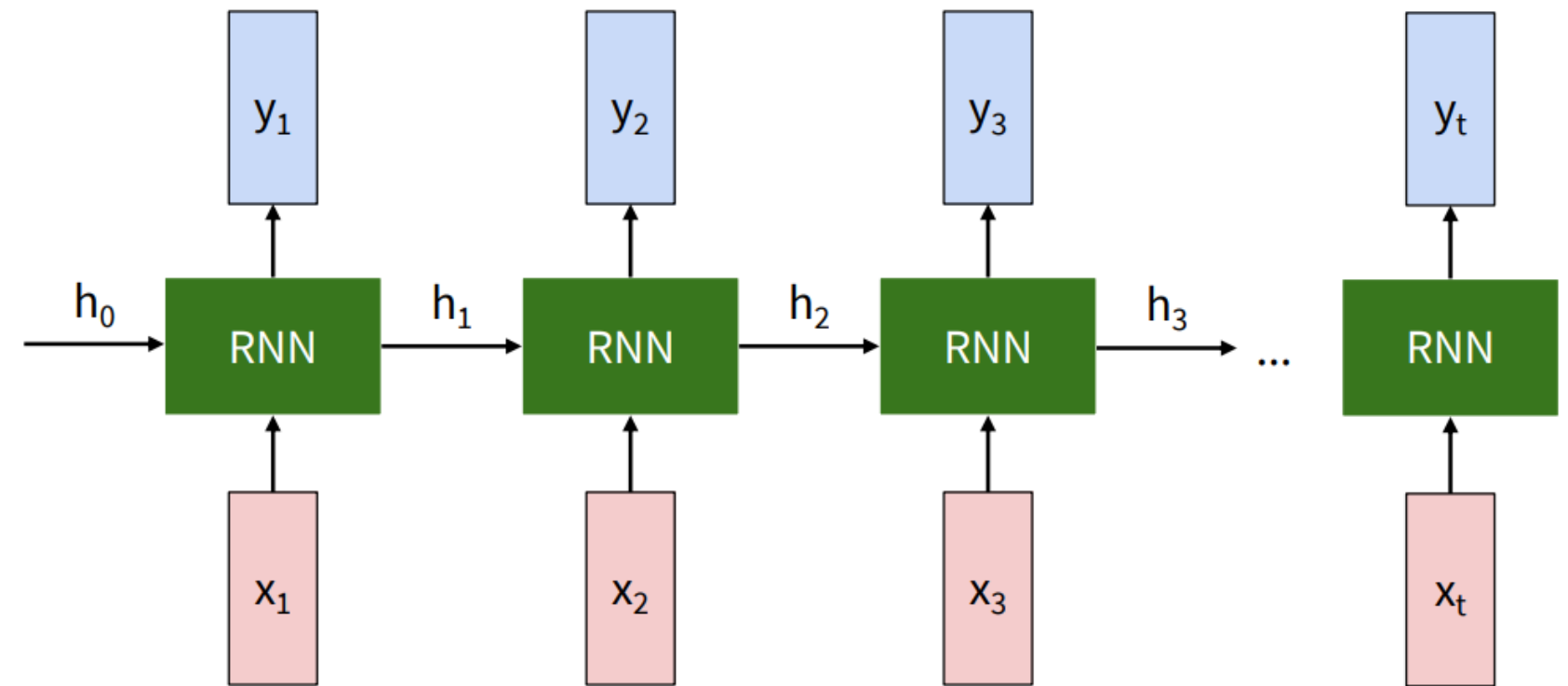
some function with parameters W

$$\boxed{y_t} = \boxed{f_{W_{hy}}}(\boxed{h_t})$$

output

new state

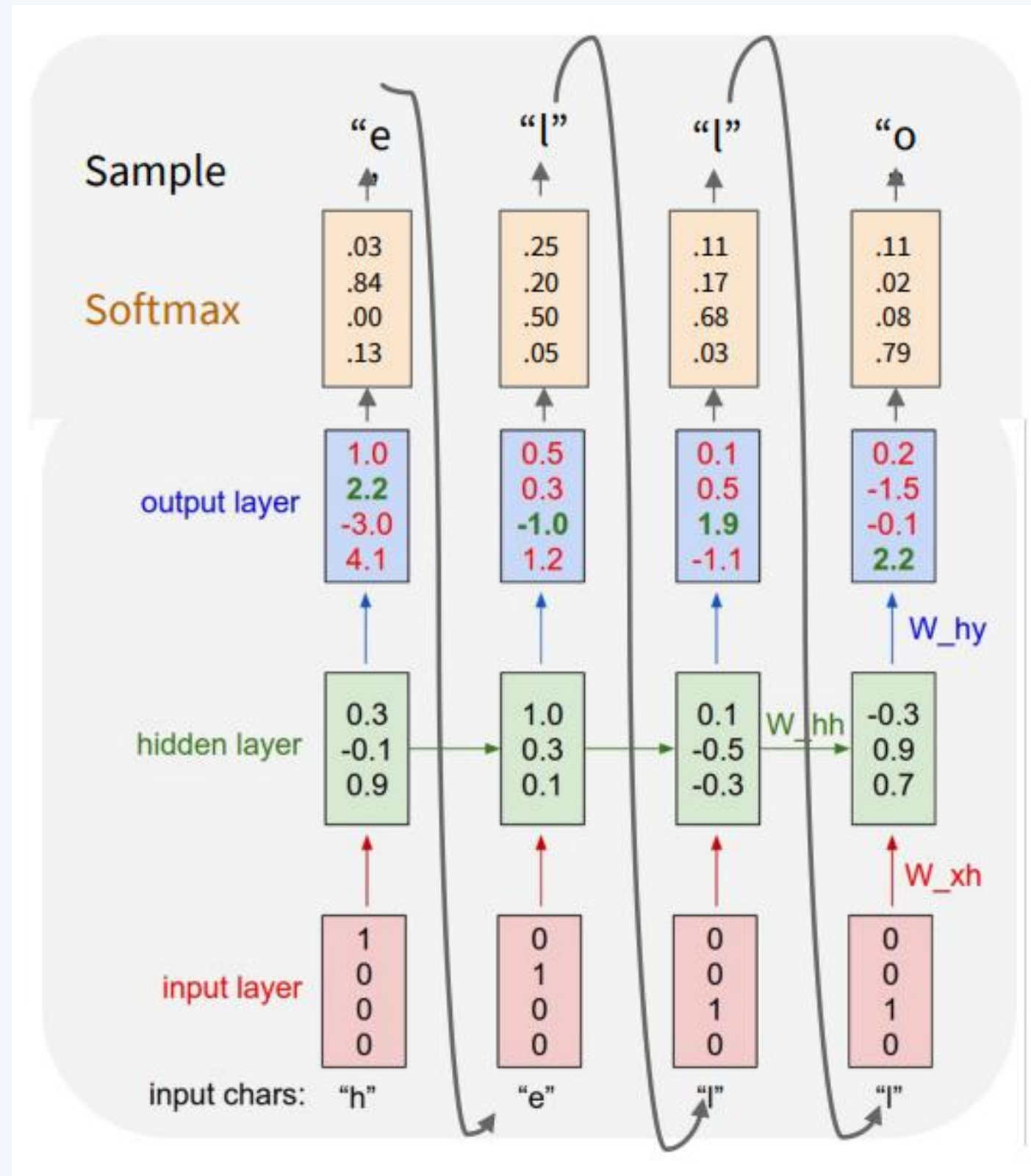
another function with parameters W_{hy}



Note:

1. RNN involves **hidden state (h)** which initialized randomly
2. At sequence, **the “h” is updated** accordingly to the function operations of the input (x) and the previous (h) given weigh (W).
3. The current “h” will be input for the output (y) and the next “h”

Case in Character Seq.

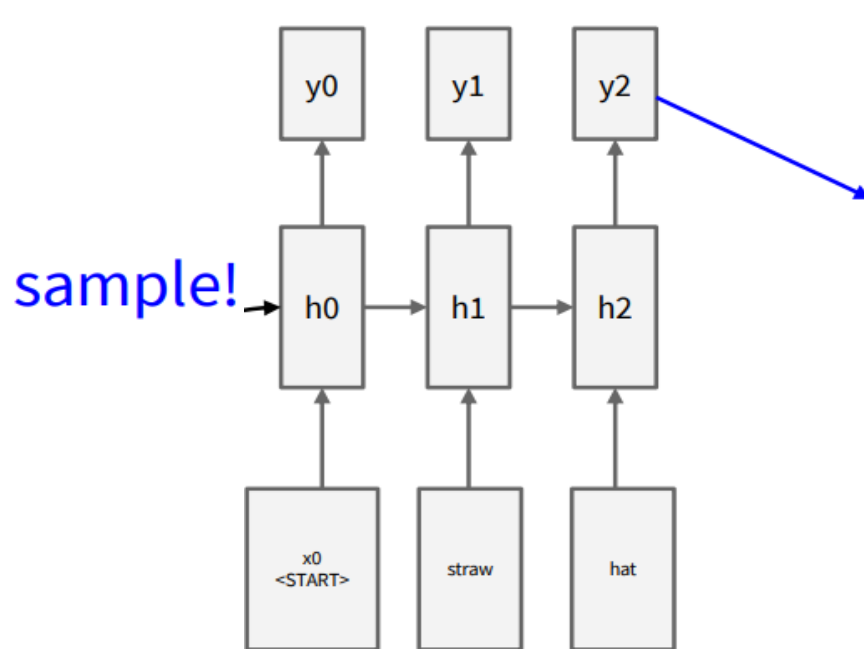
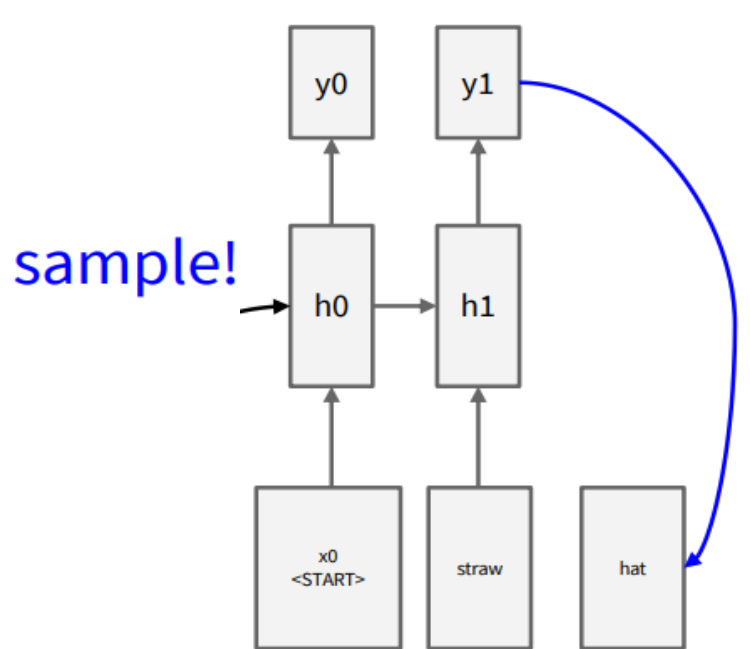
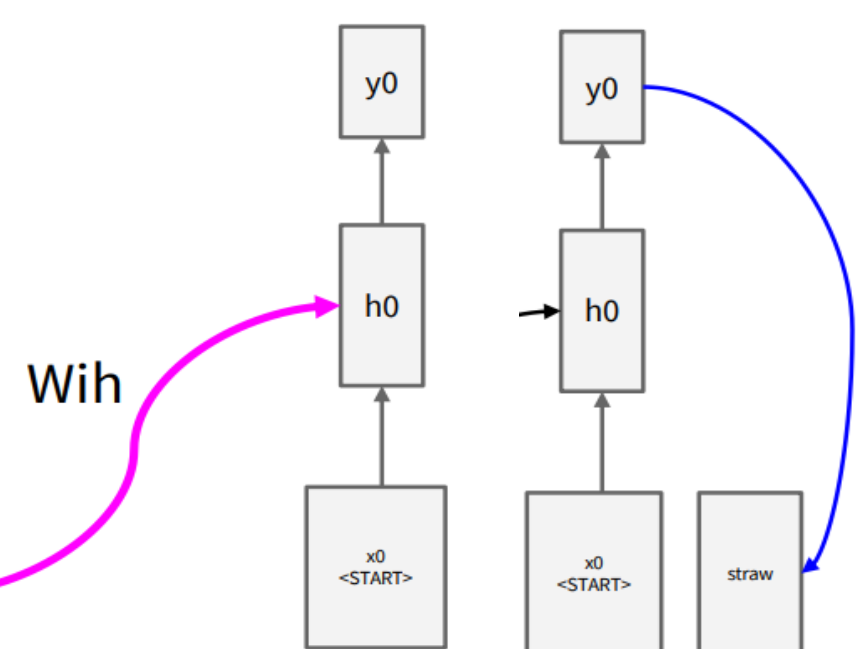


Note:

1. The "hello" word consist of "h" "e" "l" "l" "o"
2. For each char, create the vectorization $h=[1,0,0,0]$; $e=[0,1,0,0]$, $l=[0,0,1,0]$, $o=[0,0,0,1]$
3. Now given the input h, perform the rnn operation to get the output
4. Pass it to the activation layer (softmax)
5. Check the vectorization of the results $[0,1,0,0]$ which is "e"



test image



sample
<END> token
=> finish.

$$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$$

RNN, LSTM

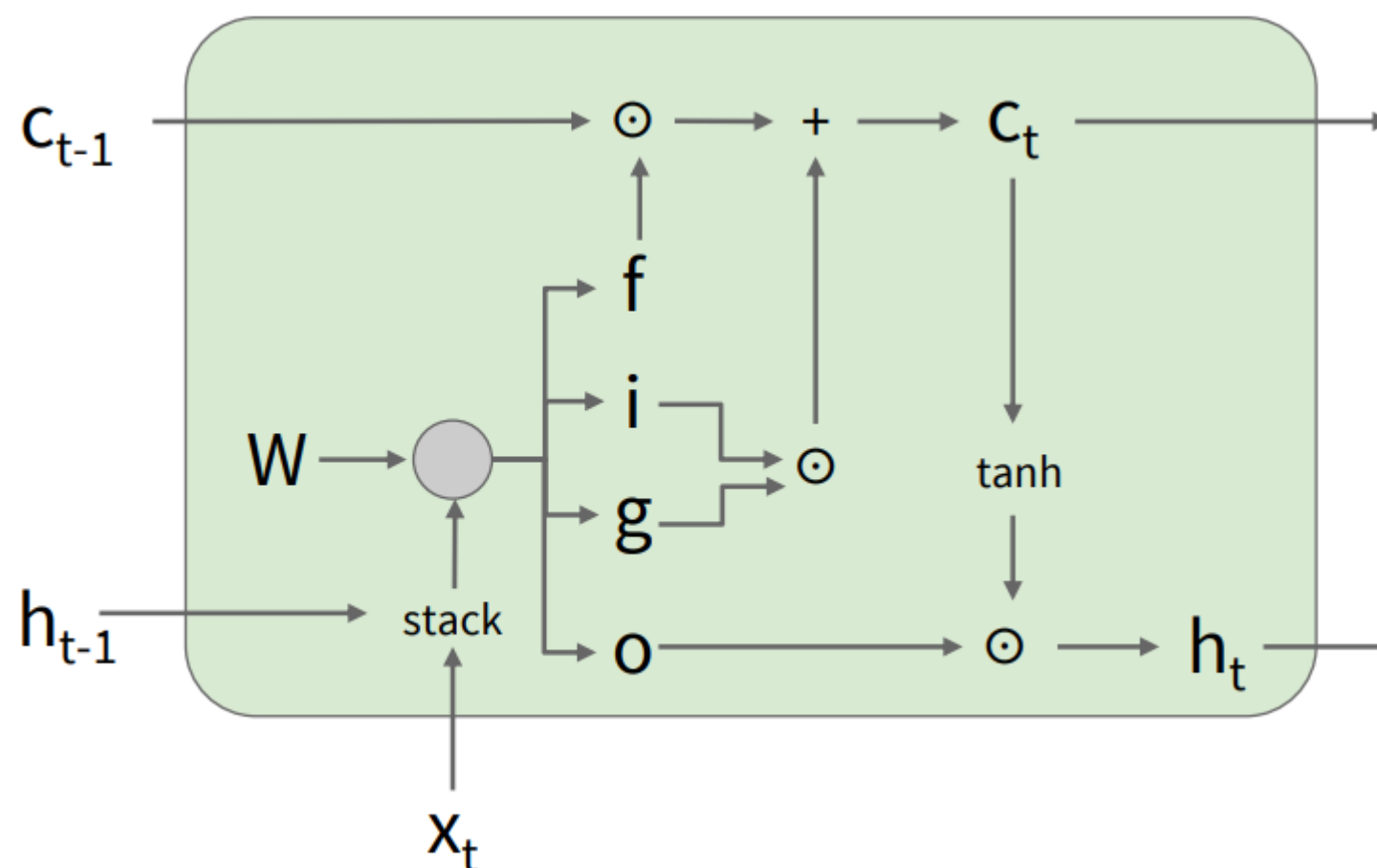


Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

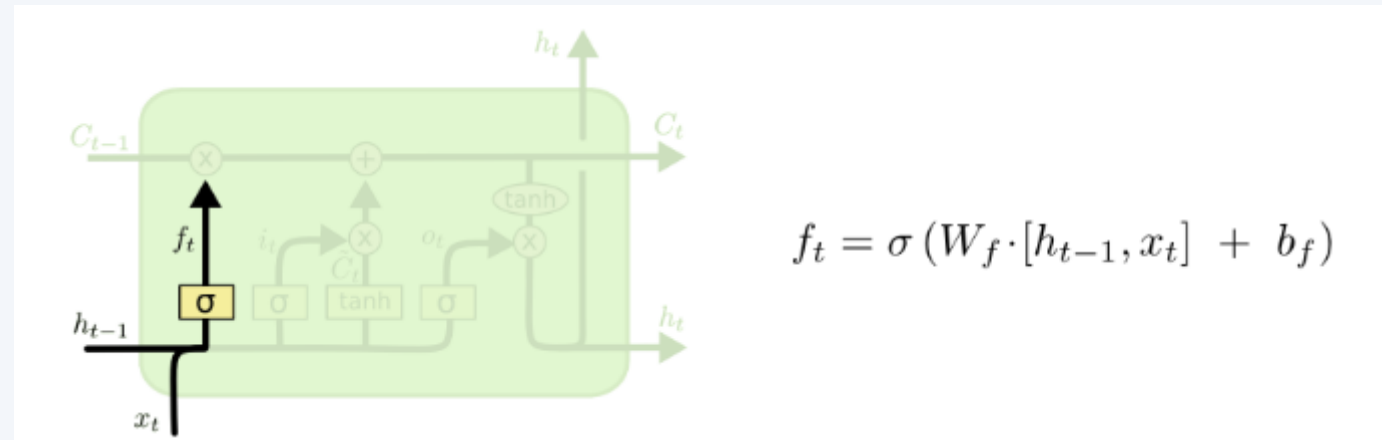
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$



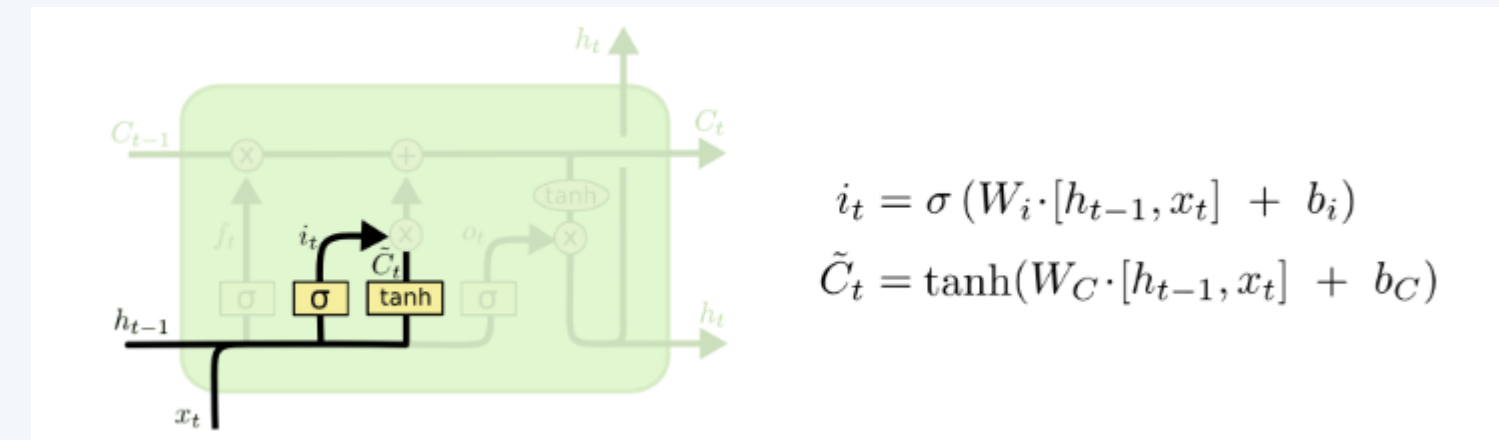
RNN, LSTM



Forget Gate – Decide What to Discard

- Input: Previous hidden state (h_{t-1}) and current input (x_t).
- Activation: Sigmoid (σ), outputs values in $[0, 1]$.
- Purpose: Decide **which parts of the previous cell state C_{t-1} to keep or forget**.
- Output: Forget vector f_t .

✅ "Should we keep or forget old information?"

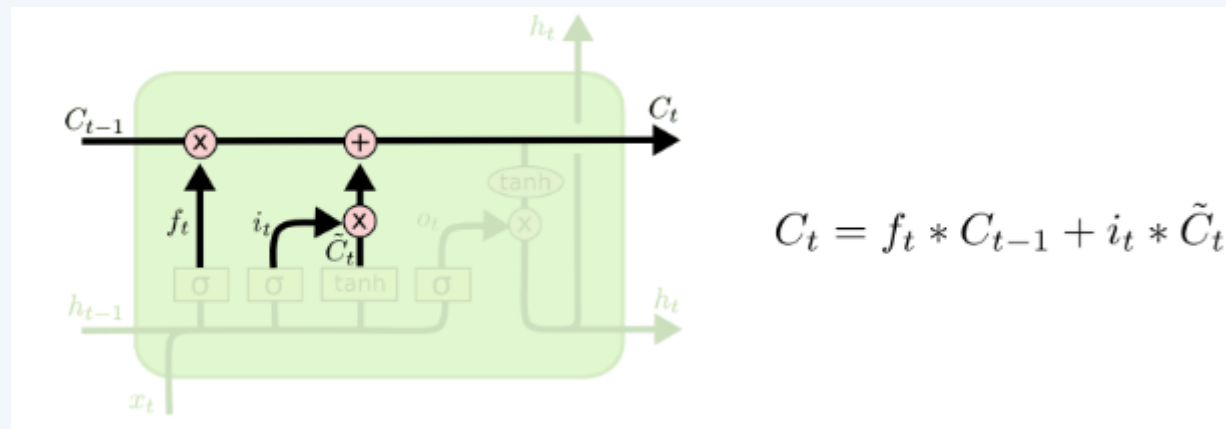


Input Gate – Decide What to Add

- Two parts:
 - **Sigmoid layer (i_t):** Decides **which values to update**.
 - **Tanh layer ($\tilde{C}_t$):** Proposes **new candidate values**.
- Purpose: Choose and prepare new information to store in the cell state.

✅ "What new information do we want to store?"

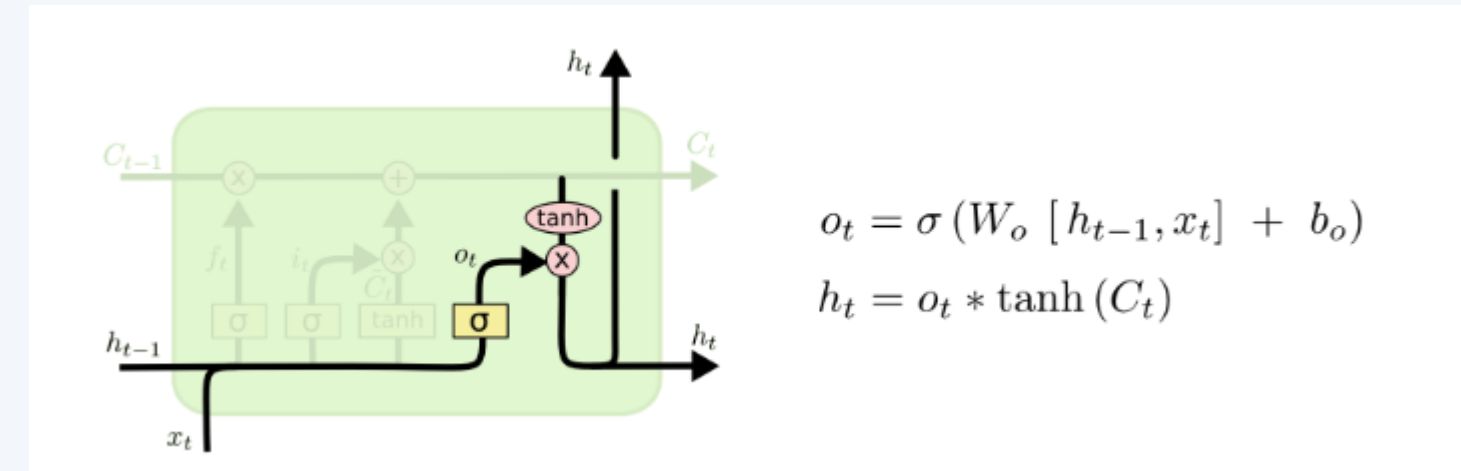
RNN, LSTM



Update Cell State

- Formula: $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$
- Action:
 - Forget old information ($f_t * C_{t-1}$).
 - Add new information ($i_t * \tilde{C}_t$).
- Purpose: Combine decisions from forget and input gates to update the **long-term memory** (C_t).

✅ "Combine old and new knowledge to update memory."

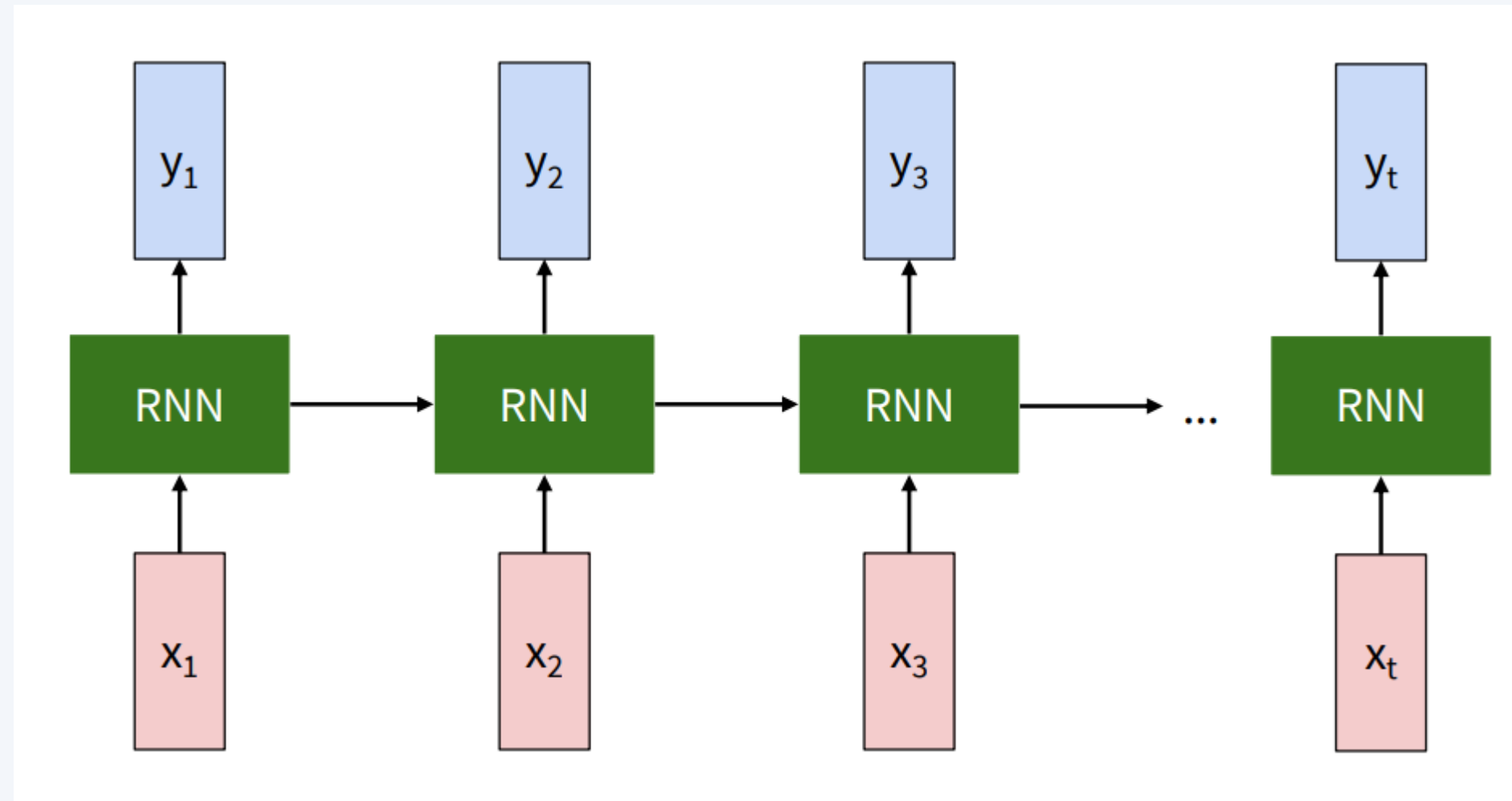


Output Gate – Decide What to Output

- Two steps:
 - Sigmoid layer: Decides **which parts of the cell state to output**.
 - Tanh + elementwise multiply: Filters the cell state through tanh and scales it with sigmoid output.
- Output: Updated hidden state h_t .

✅ "What information do we expose to the next layer or time step?"

Explain More



Click this link <https://damienox0023.github.io/rnnExplainer/> !

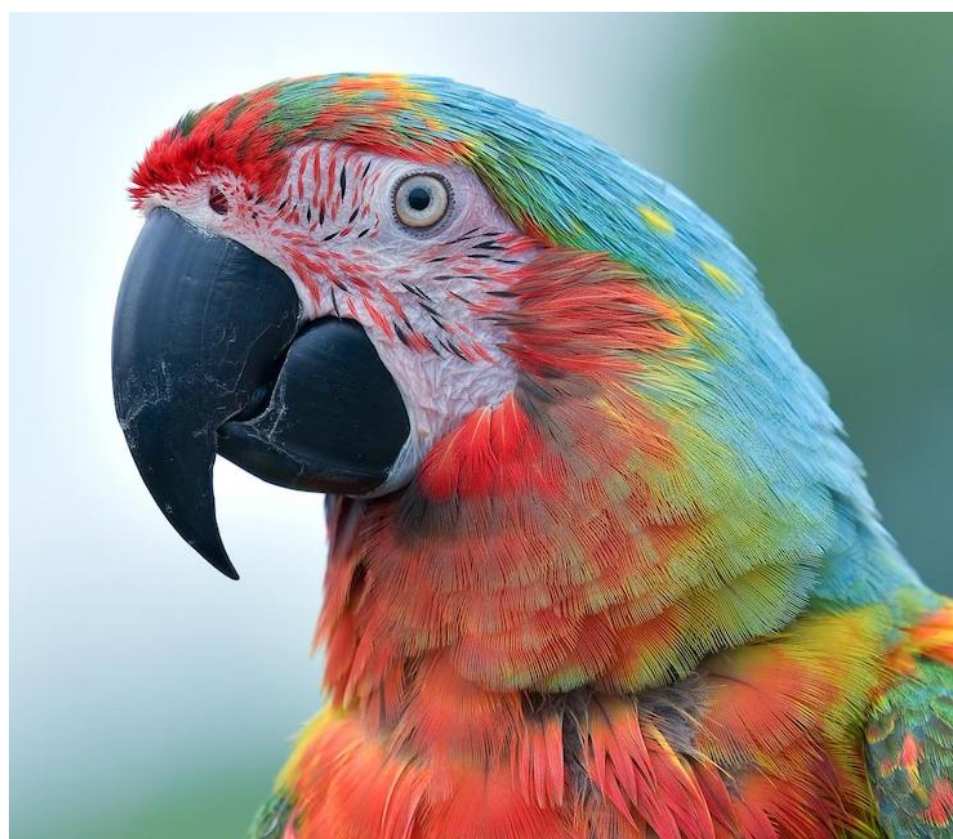
Learn more



picture	preds
	a red double decker bus driving down a street
	a group of people standing around a table
	a group of motorcycles parked on a sidewalk street

Check my work on my Master Course:

https://github.com/achmfirmanasyah/ImageProcessing/tree/main/02_Code/01_Notebook



Bonus



Advise for Final Project



- **Take a DEEP BREATH**

- » Calm your nerves. Starting is always the hardest part.

- **Follow your CURIOSITY**

- » Choose a topic you genuinely find interesting. It'll carry you through the tough days.

- **Discuss with EXPERTS**

- » Don't isolate yourself—ask supervisors, mentors, or practitioners for feedback and direction.

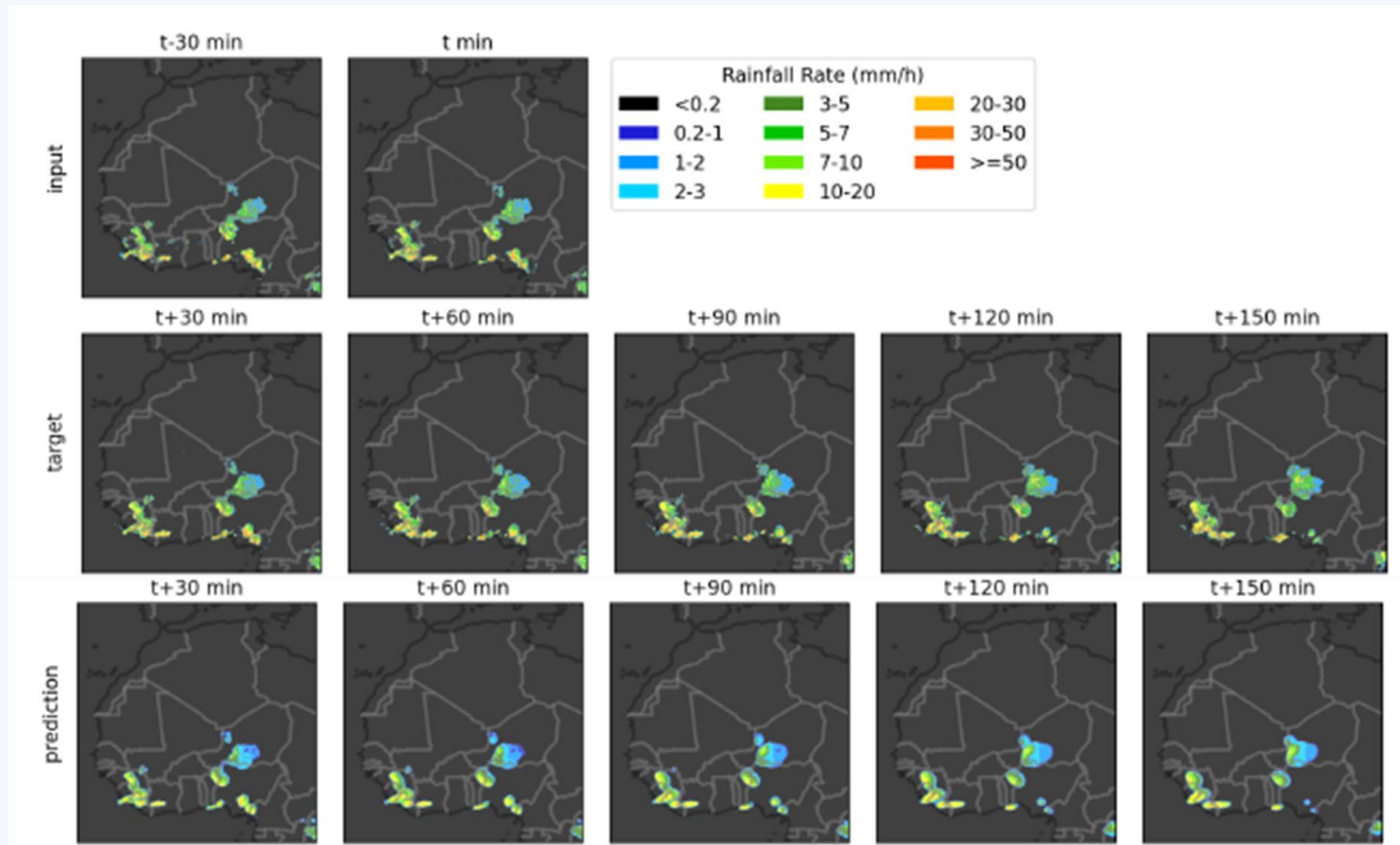
- **Set a CLEAR OBJECTIVE**

- » Define your goals early. What are you trying to solve, prove, or demonstrate?

- **REMEMBER: The BEST thesis is the FINISHED one!**

- » Perfection is the enemy of progress. Don't aim for flawless—aim for done and defensible.

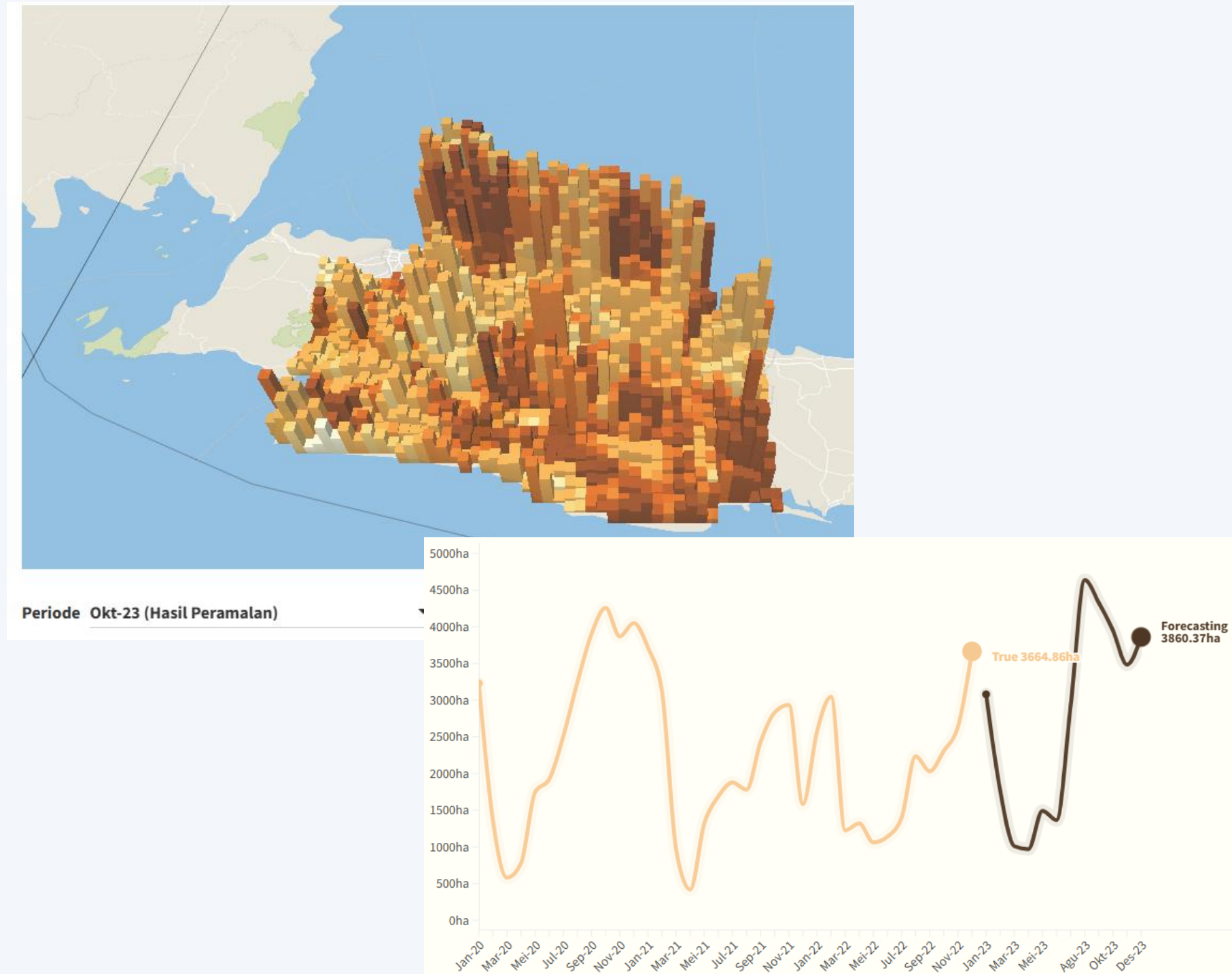
Example #1 (Rainfall Nowcasting) — ●●●



The Ideas:

- The rainfall can be monitored using satellite imagery.
- Hence, it's basically an images.
- Now, given 2 images as input? What is the next one? And how far is your accurate prediction?
- Here, I used the ConvLSTM models (Shi, 2015)
- At best scenario I can predict until next 6 hours or 12 images with only use 2 images (current and previous 30 minutes)

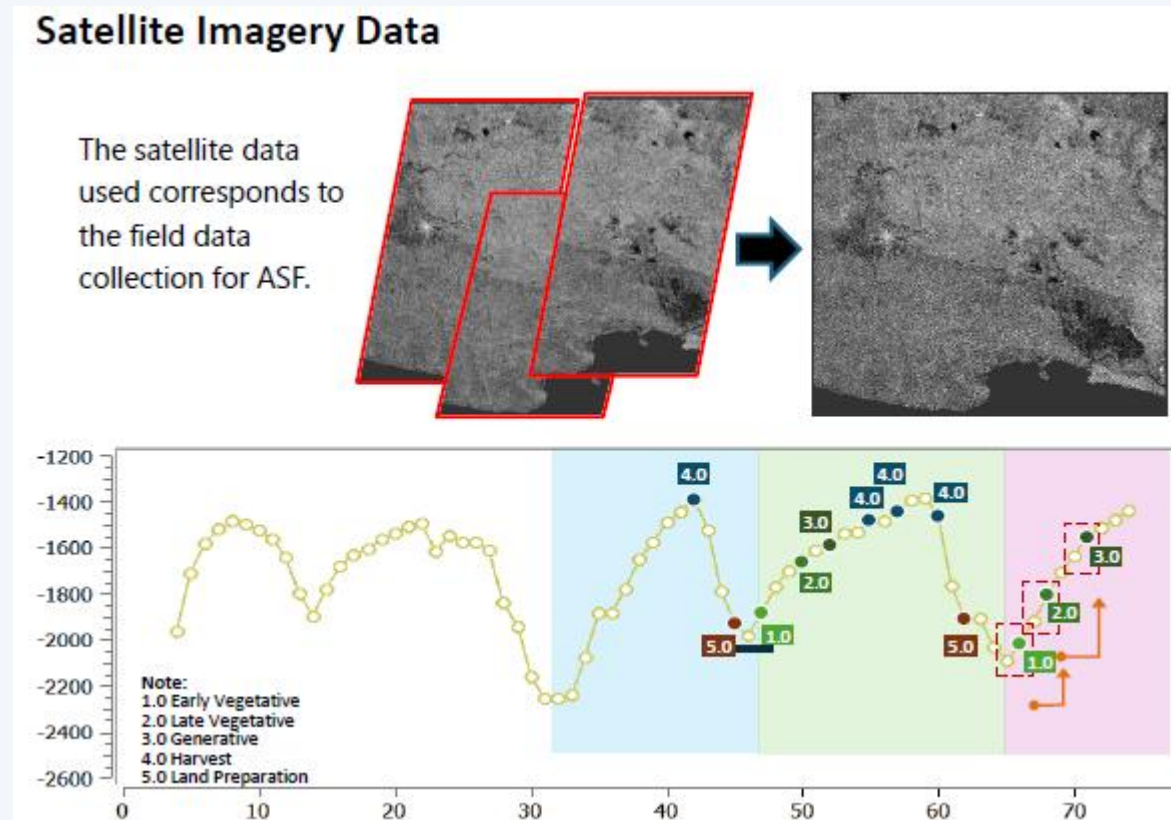
Example #2 (Drought Monitoring)— ●●●



The Ideas:

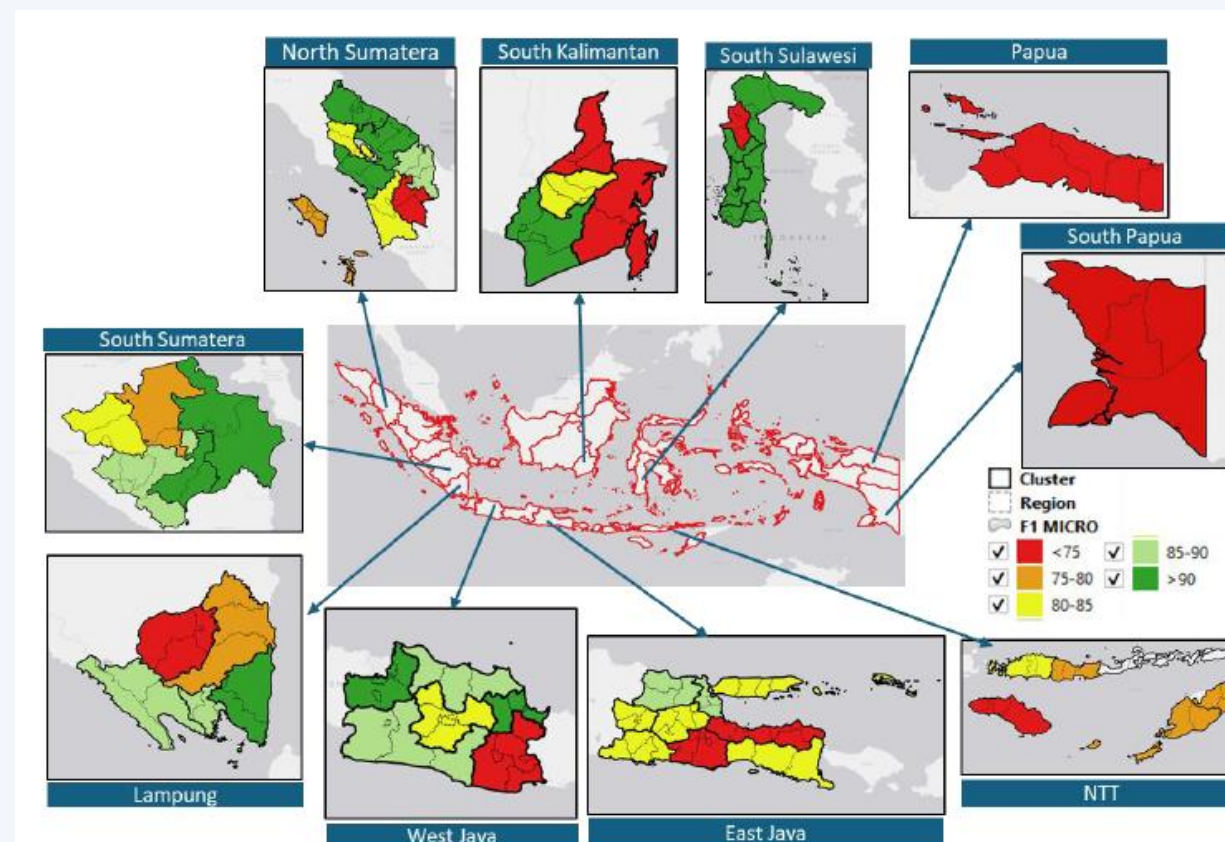
- Similar with previous idea, try to predict next drought event and the impact!
- Define the drought first → using the SDI (Severity Drought Index)
- Results: can mimic the DROUGHT phenomenon in West Java, calculate the impacted RICE FIELD, catch the EL-NINO pattern
- #1 KORIKA AI COMPETION

Example #3 (Paddy Phenology)



The Ideas:

- Use satellite imageries for predict the current paddy phenology in rice field
- Beneficial for estimating the harvested area
- Due to pattern, use the Satellite Imagery Time Series (SITS) data
- EO4SDGs 2025 Award



THANKS....

